

SENG 201 – Object Oriented Programming

2024 – 2025 Fall Lectures – Week 2

Ömer Mintemur



Introduction

Sınıf adları büyük } başlar → HelloWorld
metot adları küçük } → helloWorld
identifiers \$ veya _ veya A-Z a-z
değişkenler → helloWorld
sabitle → HELLO_WORLD

int'ten float'a geçiş yok
is ife sys0 hata verir

- Netbeans tutorial is in Aybuzem
- You can get help for installing it in lab session
- Today:
 - Will learn variables
 - How to declare them
 - Primitive data types
 - Constants
 - Selections in Java.

Java Variables

- For all data you use, you have to:
 - Assign a name
 - Data type
- Data type will tell the compiler:
 - How much memory to allocate
 - How to store the data
- Java is a statically - typed language
 - It means you should always declare your variable before the usage.

Java Variables

- Java has eight primitive data types
 - *byte*
 - *short*
 - *int*
 - *long*
 - *float*
 - *double*
 - *char*
 - *boolean*

Java Variables

- How to declare a variable
 - `dataType identifier;`
- We can declare several variables at the same time
 - `dataType identifier1, identifier2, ...;`
- Some tips
 - First letter is always lowercase
 - If you have multiple words in identifier, each word after first one begins uppercase
 - `int midtermMark`

Java Variables

- What names we can not use

abstract	continue	for	new	switch
assert ^{***}	default	goto [*]	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum ^{****}	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp ^{**}	volatile
const [*]	float	native	super	while

^{*} not used

^{**} added in 1.2

^{***} added in 1.4

^{****} added in 5.0

- ***These are called keywords***

Java Variables (Integer Types)

Type	Bytes	Minimum Value	Maximum Value
byte	1	-128	128
short	2	-32,768	32,767
int	4	-2,147,438,648	2,147,438,647
long	8	-9,223,372,036,854,775,808	-9,223,372,036,854,775,808

Examples:

```
int age;  
int pages, numberOfBooks;  
short salary;  
byte midterExam;
```

Java Variables (Floating-Point Types)

Type	Bytes	Minimum Positive Non-Zero Value	Maximum Value
float	4	1.4E-45	3.4028235E38
double	8	4.9E-324	1.7976931348623157E308

Examples:

```
float salary;
```

```
double interestRate
```

Java Variables (Char Types)

Type	Bytes	Minimum Value	Maximum Value
char	2	character - Encoded as 0	character - Encoded as FFF

Examples:

```
char letter;  
char examMark;
```

Java Variables (Boolean Types)

- For boolean, we have two values
 - `true`
 - `false`

Examples:

```
boolean hasPassed;  
boolean isFull;
```

Java Variables (How to Assign)

- How to assign values

- `int age = 10;`
- `byte examMark = 80;`
- `short carSpeed = 150;`
- `long cardNumber = 123456789L;`
- `float preciseNumber = 189.213841f;`
- `double morePreciseNumber = 189.542352351;`

When using long and float, you have to specify.

Java Variables (Be Careful !!)

- Overflow and Underflow

- If your variable is too large, it will cause **overflow**
- `int value = 2147483647 + 1; // It will overflow, and it will be -2147483647`
- `int value = -2147483647 - 1; // It will overflow, and it will be 2147483647`
- If your floating-point variable is too small, it will cause **underflow**. It will be zero (0)

overflow
underflow

Java Variables (Compatible Types)

- A variable in the right column can be assigned to a variable of any type in the left column:

Data Type	Compatible Data Type
byte	byte
short	byte, short
int	byte, short, int, char
long	byte, short, int, long, char
float	float, byte, short, int, long, char
double	float, double, byte, short, int, long, char
boolean	boolean
char	char

Java Variables (To Sum Up)

Java's primitive types				
Ordered by descending upper range limit				
CATEGORIZATION	NAME	UPPER RANGE	LOWER RANGE	BITS
Floating point	double	Really huge positive	Really huge negative	64
	float	Huge positive	Huge negative	32
Integral	long	9,223,372,036,854,775,807	-9,223,372,036,854,775,808	64
	int	2,147,483,647	-2,147,483,648	32
	char	65,535	0	16
	short	32,767	-32,768	16
	byte	127	-128	8
Boolean	boolean	No numeric equivalent	true or false	1

<https://www.theserverside.com/blog/Coffee-Talk-Java-News-Stories-and-Opinions/Java-double-precision-2-decimal-places-example-float-range-math-jvm>

Java Variables (Constant)

- If you are sure that your value will not change during your program, you can use constant
 - `final dataType identifier;`
 - `final double MILETOKM = 1.6;`
- You do not have assign a value to your constant when you declare.
- However, before the usage, a value must be assigned.
- Value of a constant variable can not be changed.

Java (How to Show Something)

- To print something in Java, you can use:
 - `System.out.print()`
 - `System.out.println()` // *ln means line*
- For more advanced usage you can use:
 - `System.out.printf()`

Java (How to Show Something)

```
public static void main(String[] args) {  
    int number = 100;  
    System.out.print("My number is " + number);  
    byte byteNumber = 80;  
    System.out.print("My byte number is " + byteNumber);  
}
```

run:

My number is 100My byte number is 80BUILD SUCCESSFUL (total time: 0 seconds)

Java (How to Show Something)

```
public static void main(String[] args) {  
    int number = 100;  
    System.out.print("My number is " + number + "\n");  
    byte byteNumber = 80;  
    System.out.print("My byte number is " + byteNumber + "\n");  
}
```

run:

My number is 100

My byte number is 80

BUILD SUCCESSFUL (total time: 0 seconds)

Java (How to Show Something)

```
public static void main(String[] args) {  
    int number = 100;  
    System.out.println("My number is " + number);  
    byte byteNumber = 80;  
    System.out.println("My byte number is " + byteNumber);  
}
```

```
run:  
My number is 100  
My byte number is 80  
BUILD SUCCESSFUL (total time: 0 seconds)
```

Java (How to Show Something)



```
public static void main(String[] args) {  
    float number = 100.249f;  
    System.out.printf("My number is %f\n", number);  
    System.out.printf("My number is %10.2f\n", number);  
    System.out.printf("My number is %.10f\n", number);  
    System.out.printf("My number is %20.10f\n", number);  
}
```

%f
↳ 3, 6 ⇒ 10

%10.2f
↳ to print 10
show 2

```
run:  
My number is 100,249001  
My number is      100,25  
My number is 100,2490005493  
My number is      100,2490005493  
BUILD SUCCESSFUL (total time: 0 seconds)
```

"%3.5f", number

Java (How to Get Input)

- First, we have to define an object
 - Do not worry, it will be generic for all semester.*
 - Scanner input = new Scanner (System.in);

After entering a number,
hit enter



```
public static void main(String[] args) {  
    Scanner input = new Scanner(System.in);  
  
    System.out.print("Enter a double number: ");  
    double myNumber = input.nextDouble();  
    System.out.println("My number is " + myNumber);  
}
```

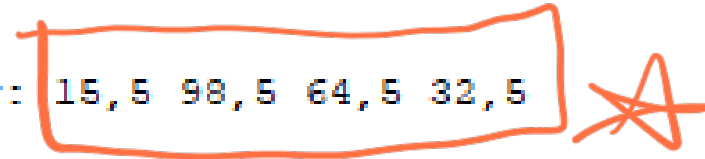
```
run:  
Enter a double number: 15,5  
My number is 15.5  
BUILD SUCCESSFUL (total time: 2 seconds)  
|
```

Scanner input = new Scanner (System.in);
double myNumber = input.nextDouble();

Java (How to Get Input)

```
public static void main(String[] args) {  
    Scanner input = new Scanner(System.in);  
  
    System.out.print("Enter a double number: ");  
    double myNumber = input.nextDouble();  
    System.out.println("My number is " + myNumber);  
  
}
```

```
run:  
Enter a double number: 15,5 98,5 64,5 32,5  
My number is 15.5  
BUILD SUCCESSFUL (total time: 14 seconds)  
|
```



Java (How to Get Input)

```
public static void main(String[] args) {  
    Scanner input = new Scanner(System.in);  
  
    System.out.print("Enter a double number: ");  
    double myNumber = input.nextDouble();  
    double mySecondNumber = input.nextDouble();  
    System.out.println("My number is " + myNumber + " My second number is " + mySecondNumber);  
}
```

run:

Enter a double number: 15,5 65,9

My number is 15.5 My second number is 65.9

BUILD SUCCESSFUL (total time: 4 seconds)

|

→ Arka arkaya alinabiliyor

Java (How to Get Input)

- How many options do we have?

nextByte()
nextShort()
nextInt()
nextFloat()
nextDouble()

Method	Description
nextByte()	Reads an integer of the byte type
nextShort()	Reads an integer of the short type
nextInt()	Reads an integer of the int type
nextFloat()	Reads a number of the float type
nextDouble()	Reads a number of the double type

Java (Selections)

- Java provides selection statements that lets you choose between two or more actions.
- A simple example will give you the idea.

```
public static void main(String[] args) {  
    Scanner input = new Scanner(System.in);  
  
    System.out.print("Please enter a number: ");  
    int myNumber = input.nextInt();  
    if (myNumber > 5) {  
        System.out.println("The number "+myNumber+ " is bigger than 5");  
    }  
}
```

```
run:  
Please enter a number: 6  
The number 6 is bigger than 5  
BUILD SUCCESSFUL (total time: 1 second)
```

Java (Selections)

- Basic syntax is:
 - `if` (condition) { }
- Operators (Assume age is 18)

Operator	Name	Example	Result
<	Less than	age < 0	
<=	Less than or equal to	age <= 0	
>	Greater than	age > 0	
>=	Greater than or equal to	age >= 0	
==	Equal to	age == 0	
!=	Not equal to	age != 0	

Java (Selections)

- We can combine it with `else`:
 - `if` (condition) { }
 - `else` {}
- Or we can combine with `else if`
 - `if` (condition) { }
 - `else if` {}

Java (Selections - Not)

- We can combine 2 or more conditions in **if** :

!
&&
||

Operator	Name
!	not
&&	and
	or

p	!p	Example (Age=20, gender='F')	Result
true	false	!(age>18)	
false	true	!(gender=='M')	

Java (Selections - And)

P1	p2	p1 && p2	Example (Age=20, gender='F')	Result
false	false	false	(age!=18) && (gender!='F')	
false	true	false	(age>25) && (gender=='F')	
true	false	false	(age>18) && (gender!='F')	
true	true	true	(age>18) && (gender=='F')	

Java (Selections - Or)

P1	p2	p1 p2	Example (Age=20, gender='F')	Result
false	false	false	(age!=18) (gender!='F')	
false	true	true	(age>25) (gender=='F')	
true	false	true	(age>18) (gender!='F')	
true	true	true	(age>18) (gender=='F')	

SENG 201 – Object Oriented Programming

2024 – 2025 Fall Lectures – Week 3

Ömer Mintemur



Introduction

- Continue with the selections
- Loops
 - For loop
 - While loop
 - Do - While loop

Selections

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    int my_number = 5;  
  
    if (my_number == 5);  
    {  
        System.out.println("My number is " + my_number  
    }  
  
    System.out.println("Run anyway");  
}
```

run:

My number is 5

Run anyway

BUILD SUCCESSFUL (total time: 0 seconds)

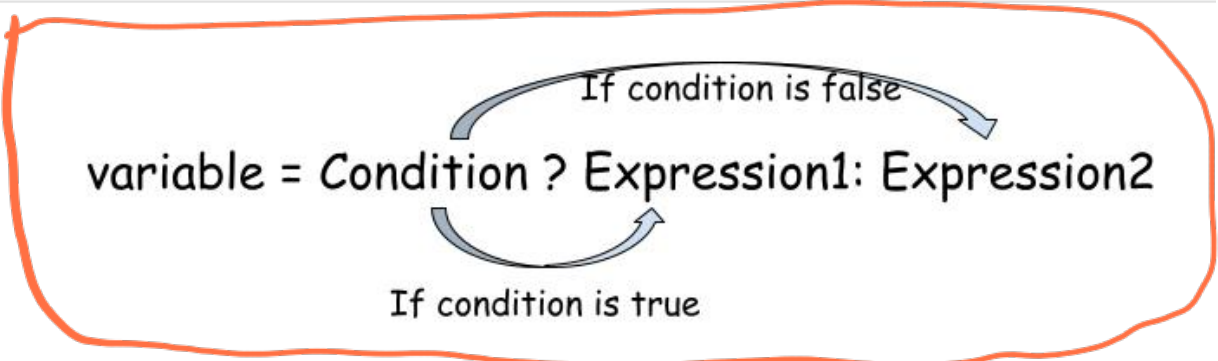
```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    int my_number = 5;  
  
    if (my_number != 5);  
    {  
        System.out.println("My number is " + my_number)  
    }  
  
    System.out.println("Run anyway");  
}
```

Selections (Ternary Operator)

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    int my_number = 5;  
    int result = 0;  
  
    if (my_number == 5)  
    {  
        result = 1;  
        System.out.println(result);  
    }  
    else  
    {  
        result = 0;  
        System.out.println(result);  
    }  
}
```

```
result = (my_number==5) ? 1:0;  
System.out.println(result);
```

variable = Condition ? Expression1: Expression2



The diagram illustrates the logic of the ternary operator. It shows the expression `variable = Condition ? Expression1: Expression2` inside a rounded rectangle. Two curved arrows originate from the `Condition` part of the expression. One arrow points to `Expression1` and is labeled "If condition is true". The other arrow points to `Expression2` and is labeled "If condition is false".

Selections (Ternary Operator)

```
public static void main(String[] args) {  
    // TODO code application logic here  
    int a = 5;  
    int b = 3;  
    int result = -1;  
    if(a==b)  
    {  
        result = 0;  
    }  
    else if(a<b)  
    {  
        result = -1;  
    }  
    else  
    {  
        result = 1;  
    }  
}
```

`result = (a==b)?0:(a<b)?-1:1;`

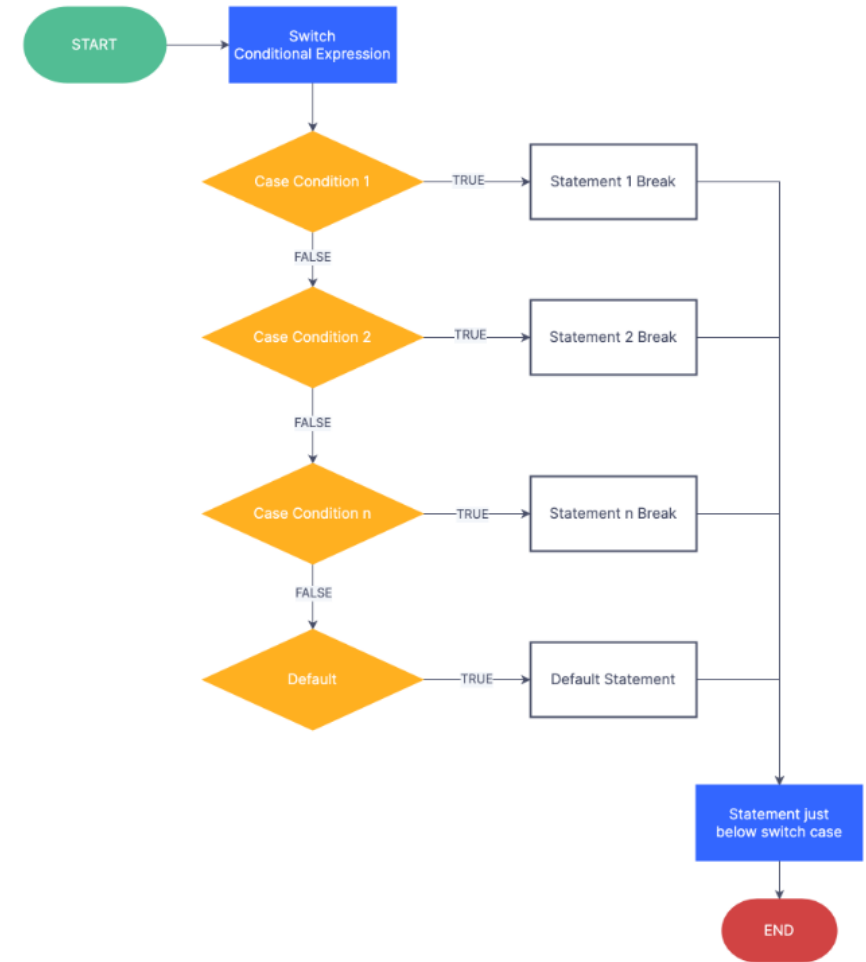
condition? ex 1 : ex 2 ;

Java Switch Case

Often used as an alternative to multiple if-else statements.

Improves code readability and maintainability.

```
public static void main(String[] args) {  
    // TODO code application logic here  
    int month = 5;  
  
    switch(month)  
    {  
        case 1: System.out.println("January");  
                break;  
        case 2: System.out.println("February");  
                break;  
        case 3: System.out.println("March");  
                break;  
        default:  
            System.out.println("Wrong Month");  
    }  
}
```



Java Switch Case – If equivalent

```
public static void main(String[] args) {  
    // TODO code application logic here  
    int month = 5;  
  
    switch(month)  
    {  
        case 1: System.out.println("January");  
                break;  
        case 2: System.out.println("February");  
                break;  
        case 3: System.out.println("March");  
                break;  
        default:  
            System.out.println("Wrong Month");  
    }  
}
```

```
if(month==1)  
{  
    System.out.println("January");  
}  
else if (month==2)  
{  
    System.out.println("February");  
}  
else if (month==3)  
{  
    System.out.println("March");  
}  
else  
{  
    System.out.println("Wrong Month");  
}
```

Java Switch Case – If equivalent

```
if(month == 1 || month == 2)
{
    System.out.println("January - February");
}
else if (month == 3 || month == 4)
{
    System.out.println("March - April");
}
else
{
    System.out.println("Wrong Month");
}
```

```
int month = 4;

switch(month)
{
    case 1:
    case 2:
        System.out.println("January - February");
        break;
    case 3:
    case 4:
        System.out.println("March - April");
        break;
    default:
        System.out.println("Wrong Month");
}
```

Java Switch Case

```
public static void main(String[] args) {  
    // TODO code application logic here  
    int month = 1;  
  
    switch(month)  
    {  
        case 1:  
        case 2:  
            System.out.println("January - February");  
        case 3:  
        case 4:  
            System.out.println("March - April");  
        default:  
            System.out.println("Wrong Month");  
    }  
}
```

```
run:  
January - February  
March - April  
Wrong Month  
BUILD SUCCESSFUL (total time: 0 seconds)
```

Your statements are executed until **break** is seen

Float or double number in if conditions (to check equality) or switch are not allowed.

Java Loops

- What is a loop?
 - A control flow statement that allows you to repeatedly execute a block of code
 - Used to perform repetitive tasks efficiently
 - Three main types of loops in Java:

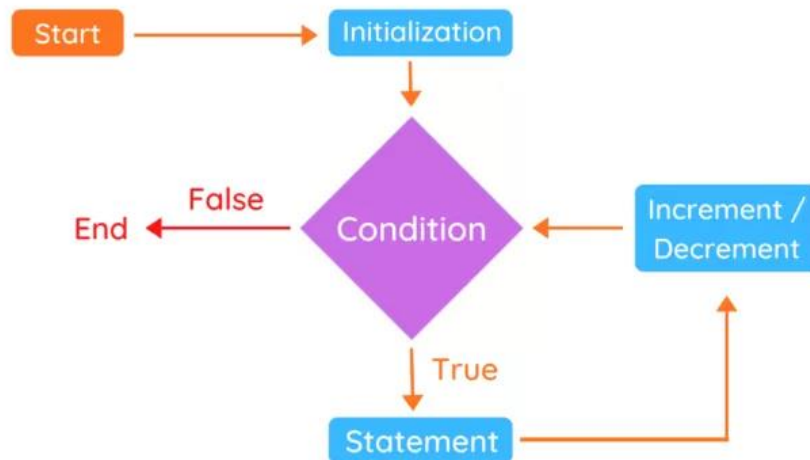
- `for`
- `while`
- `do-while`

Java For - Loops

- For loop syntax
 - `for (initialization;condition;update)`

```
for(int month=1;month<5;month++)  
{  
    System.out.println(month);  
}
```

```
run:  
1  
2  
3  
4  
BUILD SUCCESSFUL (total time: 0 seconds)  
|
```



Java For - Loops

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    int month=0;  
    for(int month=1;month<5;month++)  
    {  
        System.out.println(month);  
    }  
    for(month;month<5;month++)  
    {  
        System.out.println(month);  
    }  
}
```

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    int month=0;  
    for(;month<5;month++)  
    {  
        System.out.println(month);  
    }  
    for(;month<5;month++)  
    {  
        System.out.println(month);  
    }  
}
```

Java For - Loops

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    int first = 0;  
    int second = 0;  
    for(;first<4 && second< 7;first++,second=second+2)  
    {  
        System.out.println(first);  
        System.out.println(second);  
    }  
}
```

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    int first = 0;  
    int second = 8;  
    for(;first<4 && second< 7;first++,second=second+2)  
    {  
        System.out.println(first);  
        System.out.println(second);  
    }  
}
```

```
run:  
0  
0  
1  
2  
2  
4  
3  
6  
BUILD SUCCESSFUL (total time: 0 seconds)  
|
```

Java For - Loops

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    int first = 0;  
    for(;first<4;)  
    {  
        System.out.println(first);  
    }  
}
```

**If you do not change your variable
so that it satisfies the out condition
you can have infinite loop**



Java For - Loops

```
public static void main(String[] args) {  
    // TODO code application logic here
```

```
    int first = 0;
```

```
    for(;first<4;first++);
```

```
    {
```

```
        System.out.println(first);
```

```
    }
```

run:

4

BUILD SUCCESSFUL (total time: 0 seconds)

→ sadece sonuncuyu göster.

```
}
```

It is equivalent to

```
public static void main(String[] args) {  
    // TODO code application logic here
```

```
    int first = 0;
```

```
    for(;first<4;first++)
```

```
    {
```

```
    }
```

```
    System.out.println(first);
```

Java For - Loops

- We can break the loop so that we do not have to wait until the condition is false

We can write any condition if we are to use break

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    int first = 0;  
    for(;;first++)  
    {  
        if(first == 50)  
        {  
            break;  
        }  
    }  
    System.out.println(first);  
}
```

Java For - Loops (Nested)

- Nested loops are loops that are contained within other loops. They allow you to perform multiple levels of iteration
- Processing multi-dimensional data structures (e.g., matrices, grids)
- Generating patterns or combinations

```
for (initialization1; condition1; update1){  
    for (initialization2; condition2; update2){  
        // Code to be executed  
    }  
}
```

Java For - Loops (Nested)

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    for (int i = 1; i <= 10; i++) {  
        for (int j = 1; j <= 10; j++) {  
            System.out.print(i * j + "\t");  
        }  
        System.out.println();  
    }  
}
```

```
1 2 3 4 5 6 7 8 9 10  
2 4 6 8 10 12 14 16 18 20  
3 6 9 12 15 18 21 24 27 30  
4 8 12 16 20 24 28 32 36 40  
5 10 15 20 25 30 35 40 45 50  
6 12 18 24 30 36 42 48 54 60  
7 14 21 28 35 42 49 56 63 70  
8 16 24 32 40 48 56 64 72 80  
9 18 27 36 45 54 63 72 81 90  
10 20 30 40 50 60 70 80 90 100  
BUILD SUCCESSFUL (total time: 0 seconds)
```

Java While Loops

```
while (condition) {  
    // Code to be executed  
}
```

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    int number = 0;  
  
    while(number < 10)  
    {  
        System.out.println(number);  
        number = number + 1;  
        //number++  
    }  
  
}
```

- ✓ - First, the condition is evaluated
 - If the condition is true, the code block inside the loop is executed.
 - The condition is evaluated again, and the process repeats until the condition becomes false

Java While Loops

```
public static void main(String[] args) {  
    // TODO code application logic here  
    int first_number = 0;  
    int second_number = 0;  
    while(first_number<10 && second_number<2)  
    {  
        System.out.println("First number is "  
            + first_number +  
            " Second number is "  
            + second_number);  
        first_number++;  
        second_number++;  
    }  
}
```

run:

```
First number is 0 Second number is 0  
First number is 1 Second number is 1  
BUILD SUCCESSFUL (total time: 0 seconds)
```

```
public static void main(String[] args) {  
    // TODO code application logic here  
    int first_number = 0;  
    int second_number = 0;  
    while(first_number<10 && second_number<2);  
    {  
        System.out.println("First number is "  
            + first_number +  
            " Second number is "  
            + second_number);  
        first_number++;  
        second_number++;  
    }  
}
```

→ **Infinite Loop**

Java Do-While Loops

```
do {  
    // Code to be executed  
}
```

```
while (condition);
```

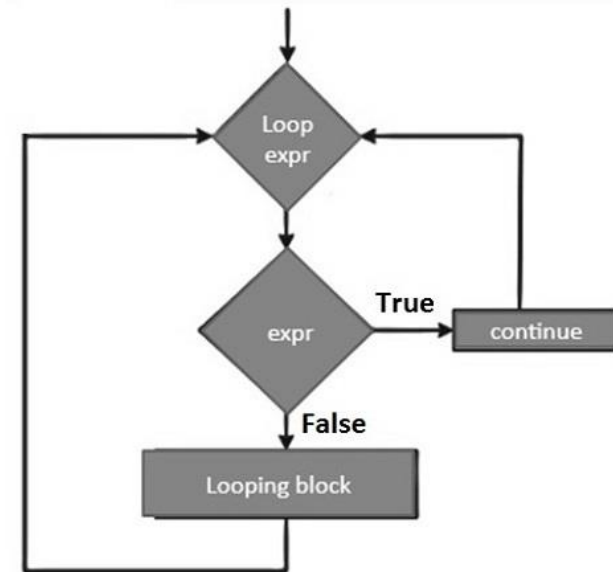
- ✓ - The code block inside the loop is executed at least once.
- The condition is then evaluated.
- If the condition is true, the loop repeats.
- If the condition is false, the loop terminates.

```
public static void main(String[] args) {  
    // TODO code application logic here  
    int i = 1;  
    do {  
        System.out.println(i);  
        i++;  
    } while (i <= 5);  
}
```

Java Loops Continue

- The `continue` statement is used to skip the current iteration of a loop and jump to the next iteration
- It is typically placed within the loop body and is often used in conjunction with conditional statements.
- When `continue` is encountered, the execution jumps back to the beginning of the loop for the next iteration

```
public static void main(String[] args) {  
    // TODO code application logic here  
    for (int i = 0; i < 10; i++) {  
        if (i % 2 == 0) {  
            continue; // Skip even numbers  
        }  
        System.out.println(i);  
    }  
}
```



Examples

Write a program that reads an integer and displays all of its factors in decreasing order. Here is the sample run

```
run:
Enter a positive integer: 30
The factors for 30 are: 30, 15, 10, 6, 5, 3, 2, 1, BUILD SUCCESSFUL (total time: 2 seconds)
|
```

Write a program that displays the following two tables side by side (1 gram is 0.035 pounds)

```
run:
Grams  pounds  pounds  Grams
1      0,035   5       142,857
3      0,105   10      285,714
5      0,175   15      428,571
7      0,245   20      571,429
9      0,315   25      714,286
11     0,385   30      857,143
13     0,455   35      1000,000
15     0,525   40      1142,857

183    6,405   460     13142,857
185    6,475   465     13285,714
187    6,545   470     13428,571
189    6,615   475     13571,429
191    6,685   480     13714,286
193    6,755   485     13857,143
195    6,825   490     14000,000
197    6,895   495     14142,857
199    6,965   500     14285,714
```

SENG 201 – Object Oriented Programming

2024 – 2025 Fall Lectures – Week 4

Ömer Mintemur



Introduction

- Single dimensional arrays
 - Search in 1-D Arrays
 - Sorting in 1-D Arrays
- 2-Dimensional Arrays
 - Accessing array elements
 - Matrix Operations

Arrays

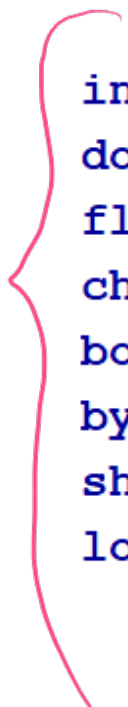
- What is an array?
 - A data structure that stores multiple elements of the same data type in a contiguous block of memory
 - Elements are accessed using an index, which starts from 0
 - Arrays have a fixed size, which cannot be changed after creation

dataType[] identifier;

```
int[] my_array={1,2,3,4};
```

```
int[] my_array = new int[4];
```

Arrays



```
int[]    int_array    = new int[4];
double[] double_array = new double[4];
float[]  float_array  = new float[4];
char[]   char_array   = new char[4];
boolean[] boolean_array = new boolean[4];
byte []  byte_array   = new byte[4];
short[]  short_array  = new short[4];
long[]   long_array   = new long[4];
```

All elements in an array must be of the same data type

```
int[] int_array = new int[4];
```

```
int_array[0] = 1;
int_array[1] = 4;
int_array[2] = 89;
int_array[3] = -10;
```

```
int[] int_array = new int[4];
```

```
int_array[0] = 1;
int_array[1] = 4;
int_array[2] = 89;
int_array[3] = 10.4;
```

Arrays

```
int[] my_array={1,2,3,4};
```

```
System.out.println(my_array);
```

Adresi bastırır

run:

[I@34a245ab

BUILD SUCCESSFUL (total time: 0 seconds)

Memory Address

```
int[] my_array={1,2,3,4};
```

```
for(int i = 0; i<4; i++)
```

```
{
```

```
    System.out.print(my_array[i]+" ");
```

```
}
```

```
System.out.println();
```

run:

1 2 3 4

BUILD SUCCESSFUL (total time: 0 seconds)

Arrays

```
int[] my_array={1,2,3,4};  
  
for(int i = 0; i<my_array.length; i++)  
{  
    System.out.print(my_array[i]+" ");  
}
```

We can directly access the number of elements of the array

```
System.out.println();
```

Arrays – ForEach Loop

```
int[] my_array={1,2,3,4};
```

```
for(int i:my_array)
{
    System.out.print(i+" ");
}
```

```
System.out.println();
```

```
for (type variable: arrayName){
    your code
}
```

for(int i : my-array)

Arrays – Search in Arrays

```
int[] my_array={30,12,89,1234,1,43,56,768};
```

```
int search = 43;
```

```
int index = 0;
```

```
for(int i:my_array)
```

```
{  
    if (i == search)  
    {  
        System.out.println("Number is in the array.");  
        System.out.println("Index is " + index);  
    }  
}
```

```
    index++;  
}
```

```
int[] my_array={30,12,89,1234,1,43,56,768};
```

```
int search = 43;
```

```
for(int i=0;i<my_array.length;i++)
```

```
{  
    if (my_array[i] == search)  
    {  
        System.out.println("Number is in the array.");  
        System.out.println("Index is " + i);  
    }  
}
```



Arrays – Sort in Arrays

- Bubble Sort
 - It works by repeatedly swapping adjacent elements if they are in the wrong order. This process is repeated until the entire array is sorted.
 - Steps:
 - Start from the beginning of the array.
 - Compare adjacent pairs of elements.
 - If the elements are in the wrong order, swap them.
 - Repeat steps 2 and 3 until no more swaps are needed.

Arrays – Sort in Arrays



```
int[] my_array={30,12,89,1234,1,43,56,768};  
for (int i = 0; i<my_array.length;i++)  
{  
    for (int y = i+1; y<my_array.length;y++)  
    {  
        if (my_array[i] > my_array[y])  
        {  
            int temp = my_array[y];  
            my_array[y] = my_array[i];  
            my_array[i] = temp;  
        }  
    }  
}
```

The code is annotated with a large orange bracket on the left side, spanning from the first 'for' loop to the end of the program. A pink box highlights the 'y = i+1' assignment in the inner loop. The swap logic (the 'if' block) is highlighted with a yellow background.

2-Dimensional Arrays

- A 2D array is a collection of elements arranged in rows and columns. It can be thought of as an array of arrays

```
int[][] myArray = new int[rows][columns];
```

```
int [] [] my_array = { {1, 2, 3}, {4, 5, 6}, {7, 8, 9} };
```

```
int [] [] my_array = { {1, 2, 3},  
                        {4, 5, 6},  
                        {7, 8, 9} };
```

2-Dimensional Arrays

```
int[][] my_array={{1,2,3},  
                 {4,5,6},  
                 {7,8,9},  
                 {10,11,12}};
```

```
System.out.println(my_array.length); // Gives number of rows  
System.out.println(my_array[0].length); // Gives number of columns
```

2-Dimensional Arrays

```
int[][] my_array={{1,2,3},  
                 {4,5,6},  
                 {7,8,9},  
                 {10,11,12}};
```

```
System.out.println("Number of rows "+my_array.length); // Gives number of rows
```

```
System.out.println("Number of columns "+my_array[0].length); // Gives number of columns
```

```
for(int x = 0;x<my_array.length;x++){  
    for(int y = 0;y<my_array[0].length;y++){  
        System.out.print(my_array[x][y]+" ");  
    }  
    System.out.println();  
}
```

2-Dimensional Arrays – Matrix Summation

$$\begin{aligned} A+B &= \begin{bmatrix} 2 & 2 & 1 \\ 1 & 5 & 0 \\ 0 & 0 & 1 \end{bmatrix} + \begin{bmatrix} 5 & 7 & 1 \\ 0 & 3 & 0 \\ 1 & 0 & 8 \end{bmatrix} \\ &= \begin{bmatrix} 2+5 & 2+7 & 1+1 \\ 1+0 & 5+3 & 0+0 \\ 0+1 & 0+0 & 1+8 \end{bmatrix} \\ &= \begin{bmatrix} 7 & 9 & 2 \\ 1 & 8 & 0 \\ 1 & 0 & 9 \end{bmatrix} \end{aligned}$$

```
int[][] first_matrix={{2,2,1},
                      {1,5,0},
                      {0,0,1},
                      };
int[][] second_matrix = {{5,7,1},
                        {0,3,0},
                        {1,0,8}
                        };
int[][] summation = new int[3][3];

for (int row = 0;row<first_matrix.length;row++){
    for(int column = 0; column<second_matrix[0].length;column++)
    {
        summation[row][column] = first_matrix[row][column] + second_matrix[row][column];
        System.out.print(summation[row][column]+ " ");
    }
    System.out.println();
}
```

2-Dimensional Arrays – Matrix Multiplication

$$\begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{bmatrix} \begin{bmatrix} b_1 & b_2 & b_3 \\ b_4 & b_5 & b_6 \\ b_7 & b_8 & b_9 \end{bmatrix} = \begin{bmatrix} c_1 & c_2 & c_3 \\ c_4 & c_5 & c_6 \\ c_7 & c_8 & c_9 \end{bmatrix}$$

```
int[][] first_matrix={{2,2,1},{1,5,0},{0,0,1},};
int[][] second_matrix = {{5,7,1},{0,3,0},{1,0,8}};
int[][] summation = {{0,0,0},{0,0,0},{0,0,0}};

for(int x = 0;x<3;x++){
    for (int y=0;y<3;y++){
        for(int t = 0;t<3;t++){
            System.out.println("[ " + x + " " + t+"]" + "++" + "[ "+t+ " " + y+"]");
            summation[x][y] += first_matrix[x][t]*second_matrix[t][y];
        }
        System.out.println("-----");
    }
}
```

0



SENG 201 – Object Oriented Programming

2024 – 2025 Fall Lectures – Week 5

Ömer Mintemur

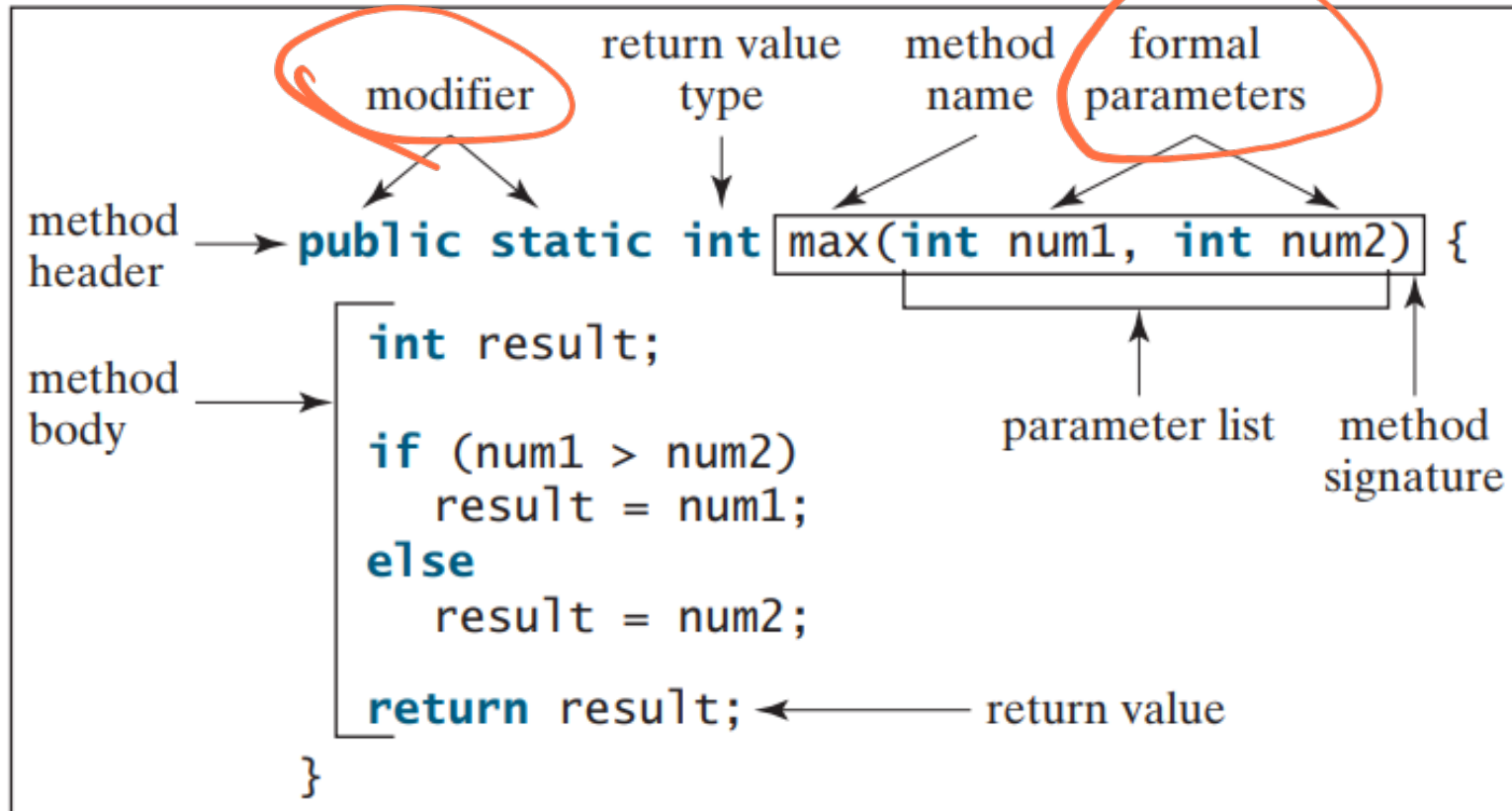


Methods - Functions

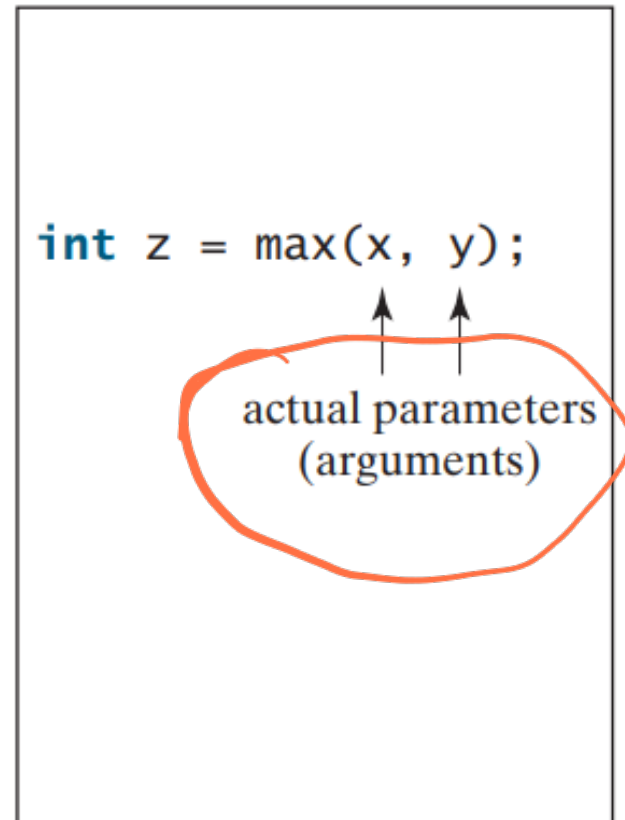
- A method in Java is a block of code that performs a specific task, defined by its name, return type, and parameters. Methods help in code reusability and modularity.
- Syntax of a Method: Methods in Java follow a specific syntax:
 - Return type (e.g., void for no return)
 - Method name (e.g., calculateSum)
 - Parameter list enclosed in parentheses (can be empty or include parameters)
 - Method body enclosed in curly braces

Methods - Functions

Define a method



Invoke a method



Methods - Functions

Specify your *return* type

```
public static void main(String[] args) {  
    // TODO code application logic here  
}
```

Write your functions after
main function

```
public static void function_name(int a, int b)  
{  
}
```

***public static is a must
until we see Objects***

Methods - Functions

If a method does not return anything, you do not need to write a return statement at the end of the function

```
public static void main(String[] args) {  
    // TODO code application logic here  
}
```

```
public static void first_function_name(int a, int b)  
{  
    int sum = a + b;  
}
```

If a method returns something, both function type and return type must match.

```
public static int second_function_name(int a, int b)  
{  
    int sum = a+b;  
    return sum;  
}
```

Methods - Functions

- A method can **return**:
 - int
 - double
 - float
 - long
 - short
 - byte
 - char
 - boolean

Methods - Functions

- Defining a function is not enough, it has to be called from the **main** function

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    first_function_name(15, 15);  
    second_function_name(15, 15);  
}  
  
public static void first_function_name(int a, int b)  
{  
    int sum = a + b;  
}  
public static int second_function_name(int a, int b)  
{  
    int sum = a+b;  
    return sum;  
}
```

Methods - Functions

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    int x = 10;  
    int y = 12;  
    first_function_name(x, y);  
    int result = second_function_name(x, y);  
  
    System.out.println(result);  
}  
  
public static void first_function_name(int a, int b)  
{  
    int sum = a + b;  
}  
  
public static int second_function_name(int a, int b)  
{  
    int sum = a+b;  
    return sum;  
}
```



```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    int x = 10;  
    int y = 12;  
    first_function_name(x, y);  
  
    System.out.println(second_function_name(x, y));  
}  
  
public static void first_function_name(int a, int b)  
{  
    int sum = a + b;  
    System.out.println(sum);  
}  
  
public static int second_function_name(int a, int b)  
{  
    int sum = a+b;  
    return sum;  
}
```


Methods - Functions

- In Java, all primitive types (e.g., `int`, `double`) are passed by value, meaning a copy of the variable is passed to the function.
- Changes made to the parameter inside the function do not affect the original variable.

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    int x = 10;  
    int y = 12;  
    System.out.println("Outside of the function x = " + x + " y = " + x);  
    change_numbers(x, y);  
    System.out.println("Outside of the function x = " + x + " y = " + x);  
}  
  
public static void change_numbers(int a, int b)  
{  
    a = 5;  
    b = 8;  
    System.out.println("Inside of the function x = " + a + " y = " + b);  
}
```

Methods - Functions

- You can not return more than one variable from a function in Java

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    int x = 10;  
    int y = 12;  
    x, y = change_numbers(x, y);  
}  
  
public static int return_numbers(int a, int b)  
{  
    a = 5;  
    b = 8;  
    return a, b;  
}
```

it-sey
return
edeneesi

Methods - Functions

- If you have an **array**, you can manipulate it in a function. Its effect will continue in the **main** function. You do not have to use **return** keyword

```
public static void main(String[] args) {  
    // TODO code application logic here
```

```
    int[] x = {10,11,12};  
    System.out.println(x[2]);  
    return_numbers(x);  
    System.out.println(x[2]);
```

```
}
```

```
public static void return_numbers(int[] a)  
{  
    a[2] = 20;  
}
```

```
run:
```

```
12
```

```
20
```

```
BUILD SUCCESSFUL (total time: 0 seconds)
```

Methods - Functions

- Function Overloading

- Function overloading allows you to define multiple functions with the same name but different parameter lists

```
run:
10
10.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

```
public static void main(String[] args) {
    // TODO code application logic here
    int x = 5;
    int y = 5;
    System.out.println(add_numbers(x,y));
    double a = 5;
    double b = 5;
    System.out.println(add_numbers(a,b));
}

public static int add_numbers(int a, int b)
{
    return a + b;
}

public static double add_numbers(double a, double b)
{
    return a + b;
}
```

Methods - Functions

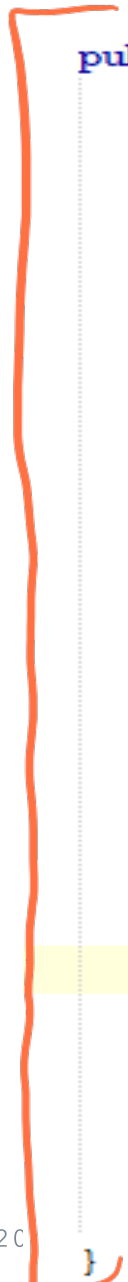
- Sometimes there are two or more possible matches for an invocation of a method, but the compiler cannot determine the most specific match.

▪ ***Overloaded methods must have different parameter lists.***

```
public static void main(String[] args) {  
    // TODO code application logic here  
    System.out.println(add_numbers(1,2));  
    System.out.println(add_numbers(1,2));  
}  
public static double add_numbers(int a, double b)  
{  
    return (a + b);  
}  
public static double add_numbers(double a, int b)  
{  
    return (a + b);  
}
```

```
public static void main(String[] args) {  
    // TODO code application logic here  
    System.out.println(add_numbers(1,2f));  
    System.out.println(add_numbers(1f,2));  
}  
public static double add_numbers(int a, float b)  
{  
    return (a + b);  
}  
public static float add_numbers(float a, int b)  
{  
    return (a + b);  
}
```

Methods – Functions – An Example



```
public static void main(String[] args) {  
    // TODO code application logic here  
    int numbers = 100;  
    int number_per_lines=10;  
    System.out.println("First " + numbers + " prime numbers are: ");  
    int count = 0;  
  
    while (numbers > 0)  
    {  
        if(isPrime(numbers))  
        {  
            count++;  
            if ((count % number_per_lines) == 0)  
            {  
                System.out.print(numbers + " \n");  
            }  
            else  
            {  
                System.out.print(numbers + " ");  
            }  
        }  
  
        numbers--;  
    }  
}
```

Methods – Functions – An Example

(Estimate π) π can be computed using the following series:

$$m(i) = 4 \left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \cdots + \frac{(-1)^{i+1}}{2i-1} \right)$$

Write a method that returns **m(i)** for a given **i** and write a test program that displays the following table:

i	m(i)
1	4.0000
101	3.1515
201	3.1466
301	3.1449
401	3.1441
501	3.1436
601	3.1433
701	3.1430
801	3.1428
901	3.1427

SENG 201 – Object Oriented Programming

2024 – 2025 Fall Lectures - Week 6

Ömer Mintemur



Objects and Classes

- Object-Oriented Programming (OOP) is a way to write programs by organizing code around objects. These objects represent real things, and they have:
 - Data (what they have)
 - Methods (what they can do)
- This makes it easier to design programs that model real-world situations.

Objects and Classes

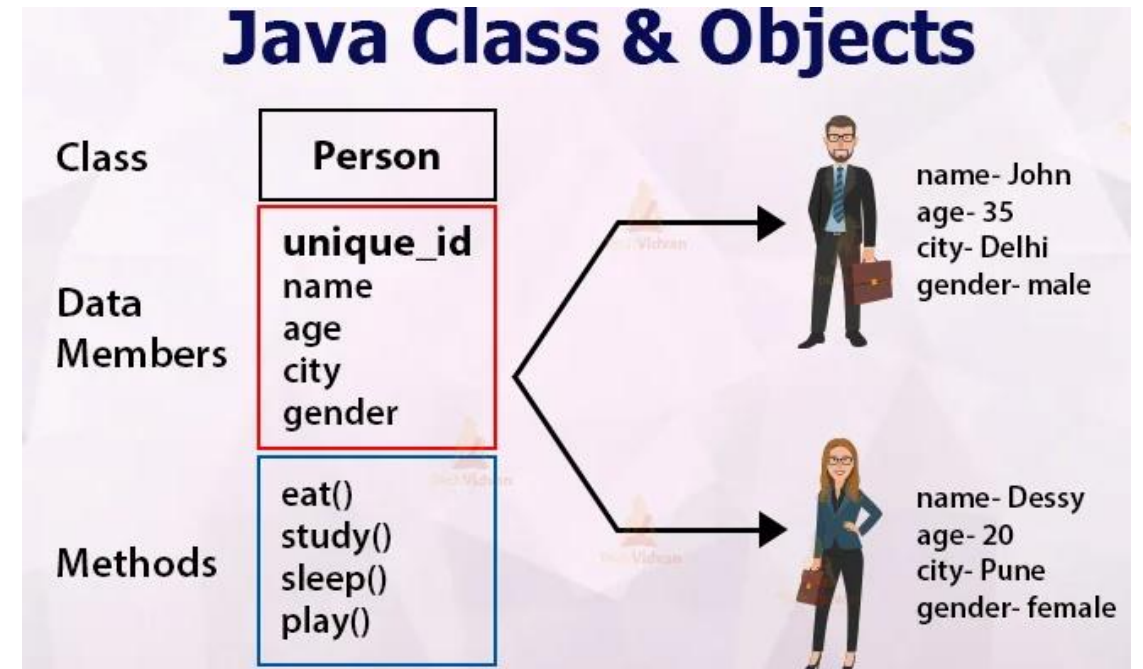
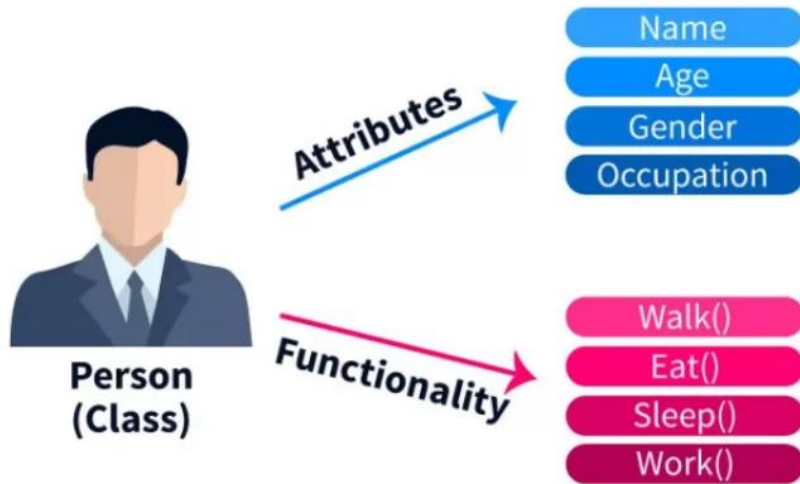
- We use classes as blueprints to create objects.
- Objects interact with each other to perform tasks in a program.
- The main principles of OOP are
 - **Encapsulation**: Hiding internal details and only exposing what's necessary
 - **Inheritance**: Reusing code by creating new classes based on existing ones
 - **Polymorphism**: Allowing objects to be used in different ways through a common interface
 - **Abstraction**: Focusing on essential features, while hiding complex details

Objects and Classes

- In Object-Oriented Programming (OOP), objects are basic units that combine data (attributes or properties) and methods (functions or behaviors) that operate on that data.
- Think of objects as real-world items, like a car. A car object would have:
 - **Attributes** (data): like color, brand, or speed.
 - **Methods** (behaviors): like drive(), stop(), or honk().
- Objects are created from classes, which act as blueprints defining the structure and behavior for the objects.

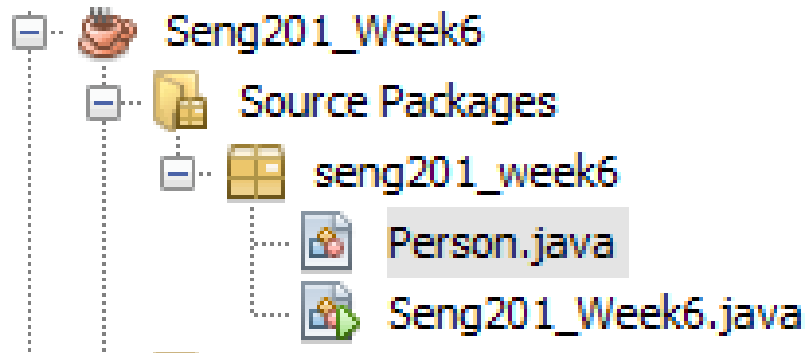
Objects and Classes

- Think of a Person. What can be the attributes and the functionalities?



Objects and Classes

- To create Objects, first we must have **Classes** in Java.
- Think of a Person again. We said that a Person has
 - Name, Gender, Age, Occupation as Attributes
 - Walk, Eat, Sleep, Work as Functionalities



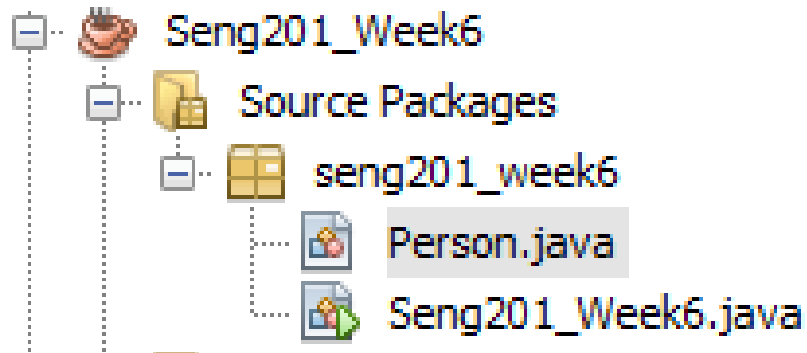
```
/**
 *
 * @author OMER
 */

public class Person {

}
```

Objects and Classes

- To create Objects, first we must have **Classes** in Java.
- Think of a Person again. We said that a Person has
 - Name, Gender, Age, Occupation as Attributes
 - Walk, Eat, Sleep, Work as Functionalities



```
/**
 *
 * @author OMER
 */

public class Person {

}
```

Objects and Classes

```
public class Person {  
  
    String name;  
    int age;  
    String gender;  
    String occupation;  
  
}
```

Attributes

Attributes and Functions

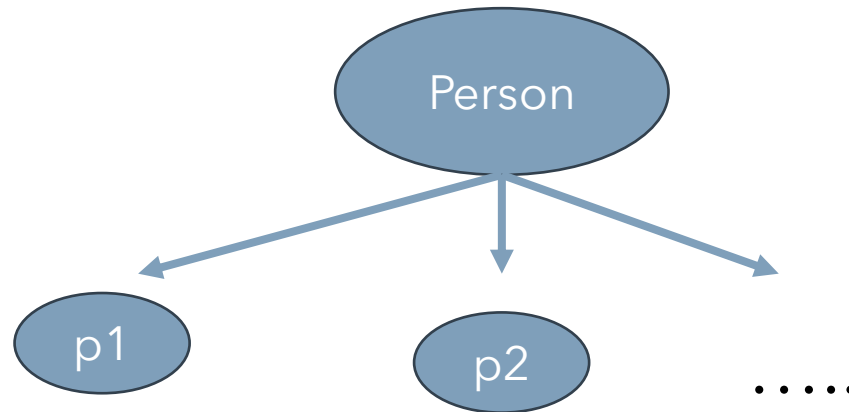
```
public class Person {  
    String name;  
    int age;  
    String gender;  
    String occupation;  
    public void Eat()  
    {  
        System.out.println("I am Eating");  
    }  
    public void Sleep()  
    {  
        System.out.println("I am Sleeping");  
    }  
    public void Walk()  
    {  
        System.out.println("I am Walking");  
    }  
    public void Work()  
    {  
        System.out.println("I am Working");  
    }  
}
```

Objects and Classes

- How do we access our classes?
- We must make an **instance** of our class in our **main file**.

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    Person p1 = new Person();  
}
```

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    Person p1 = new Person();  
    Person p2 = new Person();  
}
```



Person p1 = new Person();

Objects and Classes

- How do we use our instances?

p1.	
}	
	age int
	gender String
	name String
	occupation String
	Eat() void
	Sleep() void
	Walk() void
	Work() void

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    Person p1 = new Person();  
    Person p2 = new Person();  
  
    p1.Eat();  
    p2.Eat();  
}
```

run:

I am Eating

I am Eating

BUILD SUCCESSFUL (total time: 0 seconds)

Objects and Classes

- Notice that we did not set any attributes.

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    Person p1 = new Person();  
    Person p2 = new Person();  
  
    System.out.println(p1.age);  
    System.out.println(p1.name);  
}
```



Notice that these are functions.
It means we can modify

```
run:  
0  
null  
BUILD SUCCESSFUL (total time: 0 seconds)
```

Objects and Classes - Constructors

- A constructor in Java is a special method used to initialize objects when they are created, setting their initial state or values for the object's attributes
- **A constructor has the same name as the class and does not have a return type, not even void.**
- Constructors can be parameterized (accepting values to initialize the object's attributes) or default (without parameters, providing default values)

```
public class Person {  
    String name;  
    int age;  
    String gender;  
    String occupation;  
  
    Person(String p_name, int p_age, String p_gender, String p_occupation)  
    {  
        name = p_name;  
        age = p_age;  
        gender = p_gender;  
        occupation = p_occupation;  
    }  
  
    public void Eat()  
    {  
        System.out.println("I am Eating");  
    }  
  
    public void Sleep()  
    {  
        System.out.println("I am Sleeping");  
    }  
  
    public void Walk()  
    {  
        System.out.println("I am Walking");  
    }  
  
    public void Work()  
    {  
        System.out.println("I am Working");  
    }  
}
```

Objects and Classes - Constructors

```
public class Person {
    String name;
    int age;
    String gender;
    String occupation;

    Person(String p_name, int p_age, String p_gender, String p_occupation)
    {
        name = p_name;
        age = p_age;
        gender = p_gender;
        occupation = p_occupation;
    }

    public void Eat()
    {
        System.out.println("I am Eating");
    }

    public void Sleep()
    {
        System.out.println("I am Sleeping");
    }

    public void Walk()
    {
        System.out.println("I am Walking");
    }

    public void Work()
    {
        System.out.println("I am Working");
    }
}
```

```
public static void main(String[] args) {
    // TODO code application logic here

    Person p1 = new Person("Omer", 18, "Male", "Engineer");

    System.out.println(p1.age);
    System.out.println(p1.name);
}
```

run:

18

Omer

BUILD SUCCESSFUL (total time: 0 seconds)

Objects and Classes - Constructors

```
Person p2 = new Person();
```

```
Person()
```

```
System.out.println(p1
```

```
Person(String p_name)
```

```
System.out.println(p1
```

```
Person(String p_name, int p_age, String p_gender, String p_occupation)
```

```
public class Person {  
    String name;  
    int age;  
    String gender;  
    String occupation;  
    Person()  
    {  
  
    }  
    Person(String p_name)  
    {  
        name = p_name;  
    }  
    Person(String p_name, int p_age, String p_gender, String p_occupation)  
    {  
        name = p_name;  
        age = p_age;  
        gender = p_gender;  
        occupation = p_occupation;  
    }  
}
```

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    Person p1 = new Person("Omer", 18, "Male", "Engineer");  
    Person p2 = new Person("Hakan");  
    Person p3 = new Person();  
}
```

Objects and Classes - Constructors

```
Person() {}  
Person(String p_name)  
{  
    name = p_name;  
}  
Person(String p_name, int p_age, String p_gender, String p_occupation)  
{  
    name = p_name;  
    age = p_age;  
    gender = p_gender;  
    occupation = p_occupation;  
}  
public void PersonInfo()  
{  
    System.out.println("Person's name is: "+name);  
    System.out.println("Person's age is: "+age);  
    System.out.println("Person's gender is: "+gender);  
    System.out.println("Person's occupation is: "+occupation);  
}
```

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    Person p1 = new Person("Omer", 18, "Male", "Engineer");  
    p1.PersonInfo();  
}
```

```
run:  
Person's name is: Omer  
Person's age is: 18  
Person's gender is: Male  
Person's occupation is: Engineer  
BUILD SUCCESSFUL (total time: 0 seconds)
```

Objects and Classes

We can directly access and changes the attributes from our main program

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    Person p1 = new Person("Omer", 18, "Male", "Engineer");  
  
    p1.age = 60;  
    p1.name = "John";  
  
    p1.PersonInfo();  
}
```

```
run:  
Person's name is: John  
Person's age is: 60  
Person's gender is: Male  
Person's occupation is: Engineer  
BUILD SUCCESSFUL (total time: 0 seconds)
```

However, we should not be able to access the attributes of a class from our main function.

Objects and Classes – Private Keyword

We have to use **private** keyword for our attributes, so that they can not be accessed from outside of the class.

```
public class Person {  
  
    private String name;  
    private int age;  
    private String gender;  
    private String occupation;  
}
```

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    Person p1 = new Person("Omer", 18, "Male", "Engineer");  
  
    p1.age = 60;  
    p1.name = "John";  
  
    p1.PersonInfo();  
}
```


Objects and Classes – Getters and Setters

- So, how do we access our attributes?
- For accessing attributes, we have to use **Getters and Setters**
- Setters provide flexibility by allowing you to control and validate the values being assigned to your object's attributes.

```
public class Person {  
  
    private String name;  
    private int age;  
    private String gender;  
    private String occupation;  
  
    public void setName(String p_name)  
    {  
        name = p_name;  
    }  
    public void setAge(int p_age)  
    {  
        age = p_age;  
    }  
    public void setGender(String p_gender)  
    {  
        gender = p_gender;  
    }  
    public void setOccupation(String p_occupation)  
    {  
        occupation = p_occupation;  
    }  
}
```

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    Person p1 = new Person("Omer", 18, "Male", "Engineer");  
  
    p1.setName("John");  
    p1.setAge(60);  
    p1.PersonInfo();  
}
```

Objects and Classes – Getters and Setters

- Getters simply return the value of an object's attribute, allowing access to its data without modifying it.

```
public class Person {  
  
    private String name;  
    private int age;  
    private String gender;  
    private String occupation;  
    public String getName()  
    {  
        return name;  
    }  
    public int getAge()  
    {  
        return age;  
    }  
    public String getGender()  
    {  
        return gender;  
    }  
    public String getOccupation()  
    {  
        return occupation;  
    }  
}
```

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    Person p1 = new Person("Omer", 18, "Male", "Engineer");  
  
    p1.setName("John");  
    p1.setAge(60);  
    System.out.println(p1.getName());  
    System.out.println(p1.getAge());  
}
```

```
run:  
John  
60  
BUILD SUCCESSFUL (total time: 0 seconds)
```

SENG 201 – Object Oriented Programming

2024 - 2025 Fall Lectures - Week 8

Ömer Mintemur



Exception Handling

- What is Exception?
 - Exception is an abnormal condition that arises at runtime.
 - It is an object which is thrown at runtime.
- What is Exception Handling?
 - Mechanism to manage runtime errors and maintain normal program flow.
 - Allows you to respond to unexpected conditions and prevent program crashes.

Exception Handling Keywords

- **try** block contains the code to monitor for exceptions
 - If an exception occurs within the try block, it is thrown
- **catch** block contains code to catch and handle the exception
- **throw** is used to explicitly throw an exception
- **throws** specifies any exceptions that a method might throw
- **finally** block contains code that will always execute after the try block, regardless of whether an exception occurs

Exception Handling

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    int a = 4;  
    int b = 0;  
  
    int result = a/b;  
}
```

run:
[-] Exception in thread "main" java.lang.ArithmeticException: / by zero
 at seng201_week_8.SENG201_Week_8.main(SENG201_Week_8.java:23)
[C:\Users\OMER\AppData\Local\NetBeans\Cache\12.1\executor-snippets\run.xml:111](#): The following error occurred while executing this line:
[C:\Users\OMER\AppData\Local\NetBeans\Cache\12.1\executor-snippets\run.xml:68](#): Java returned: 1
BUILD FAILED (total time: 0 seconds)

Exception Handling

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    int a = 4;  
    int b = 0;  
    try{  
        int result = a/b;  
    }  
    finally {  
        System.out.println("This block will always execute");  
    }  
}
```

```
run:  
This block will always execute  
[Exception in thread "main" java.lang.ArithmeticException: / by zero  
[    at seng201_week_8.SENG201_Week_8.main(SENG201_Week_8.java:23)  
C:\Users\OMER\AppData\Local\NetBeans\Cache\12.1\executor-snippets\run.xml:111: The following error occurred while executing this line:  
C:\Users\OMER\AppData\Local\NetBeans\Cache\12.1\executor-snippets\run.xml:68: Java returned: 1  
BUILD FAILED (total time: 0 seconds)
```

Exception Handling

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    int a = 4;  
    int b = 2;  
    try{  
        int result = a/b;  
    }  
  
    finally {  
        System.out.println("This block will always execute");  
    }  
}
```

run:

This block will always execute

BUILD SUCCESSFUL (total time: 0 seconds)

Exception Handling

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    int a = 4;  
    int b = 0;  
    try{  
        int result = a/b;  
    }  
  
    finally {  
        System.out.println("This block will always execute");  
    }  
  
    System.out.println("Rest of the code after try-finally block");  
}
```

run:
This block will always execute
] Exception in thread "main" java.lang.ArithmeticException: / by zero
at seng201_week_8.SENG201_Week_8.main(SENG201_Week_8.java:23)
C:\Users\OMER\AppData\Local\NetBeans\Cache\12.1\executor-snippets\run.xml:111: The following error occurred while executing this line:
C:\Users\OMER\AppData\Local\NetBeans\Cache\12.1\executor-snippets\run.xml:68: Java returned: 1
BUILD FAILED (total time: 0 seconds)

→ catch you

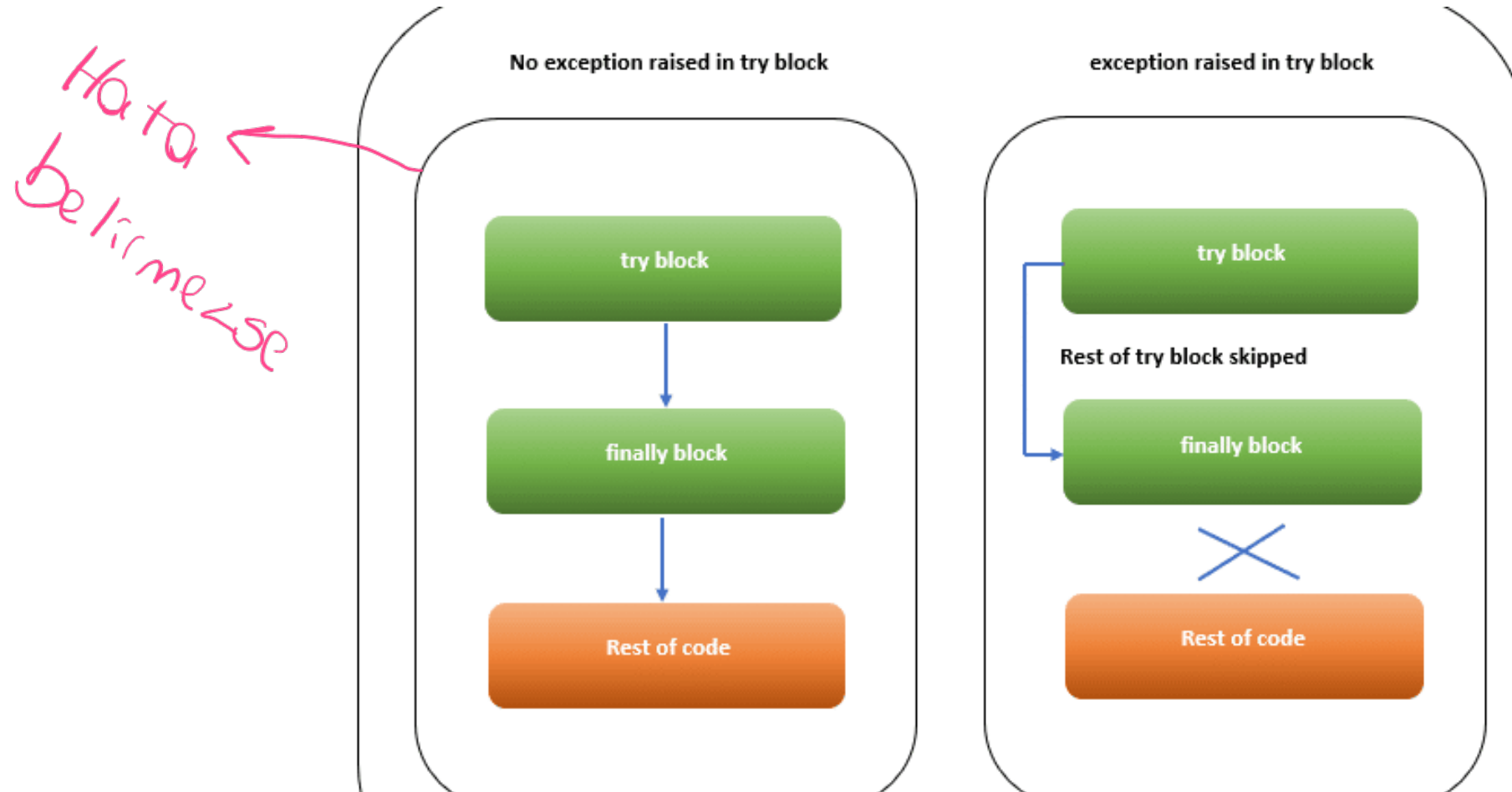
Exception Handling

```
public class SENG201_Week_8 {  
    public static void main(String[] args) {  
        // TODO code application logic here  
        try {  
            int result = 15 / 0;  
            System.out.println("Result: " + result);  
        } catch (Exception e) {  
            System.out.println("Logging: Cannot divide by zero.");  
        }  
  
        System.out.println("Continue with the main code");  
    }  
}
```

run:
Logging: Cannot divide by zero.
Continue with the main code
BUILD SUCCESSFUL (total time: 0 seconds)

catch file
devs, o/v,

Exception Handling



Exception Handling

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    int a = 4;  
    int b = 0;  
    try{  
        int result = a/b;  
    }  
    catch (Exception e)  
    {  
        System.out.println("Exception occurred " + e);  
    }  
    finally {  
        System.out.println("This block will always execute");  
    }  
  
    System.out.println("Rest of the code after try-finally block");  
}
```

run:

Exception occurred java.lang.ArithmeticException: / by zero

This block will always execute

Rest of the code after try-finally block

BUILD SUCCESSFUL (total time: 0 seconds)

Exception Handling

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    int a = 4;  
    int b = 1;  
    try{  
        int result = a/b;  
    }  
    catch (Exception e)  
    {  
        System.out.println("Exception occurred " + e);  
    }  
    finally {  
        System.out.println("This block will always execute");  
    }  
  
    System.out.println("Rest of the code after try-finally block");  
}
```

hata yok

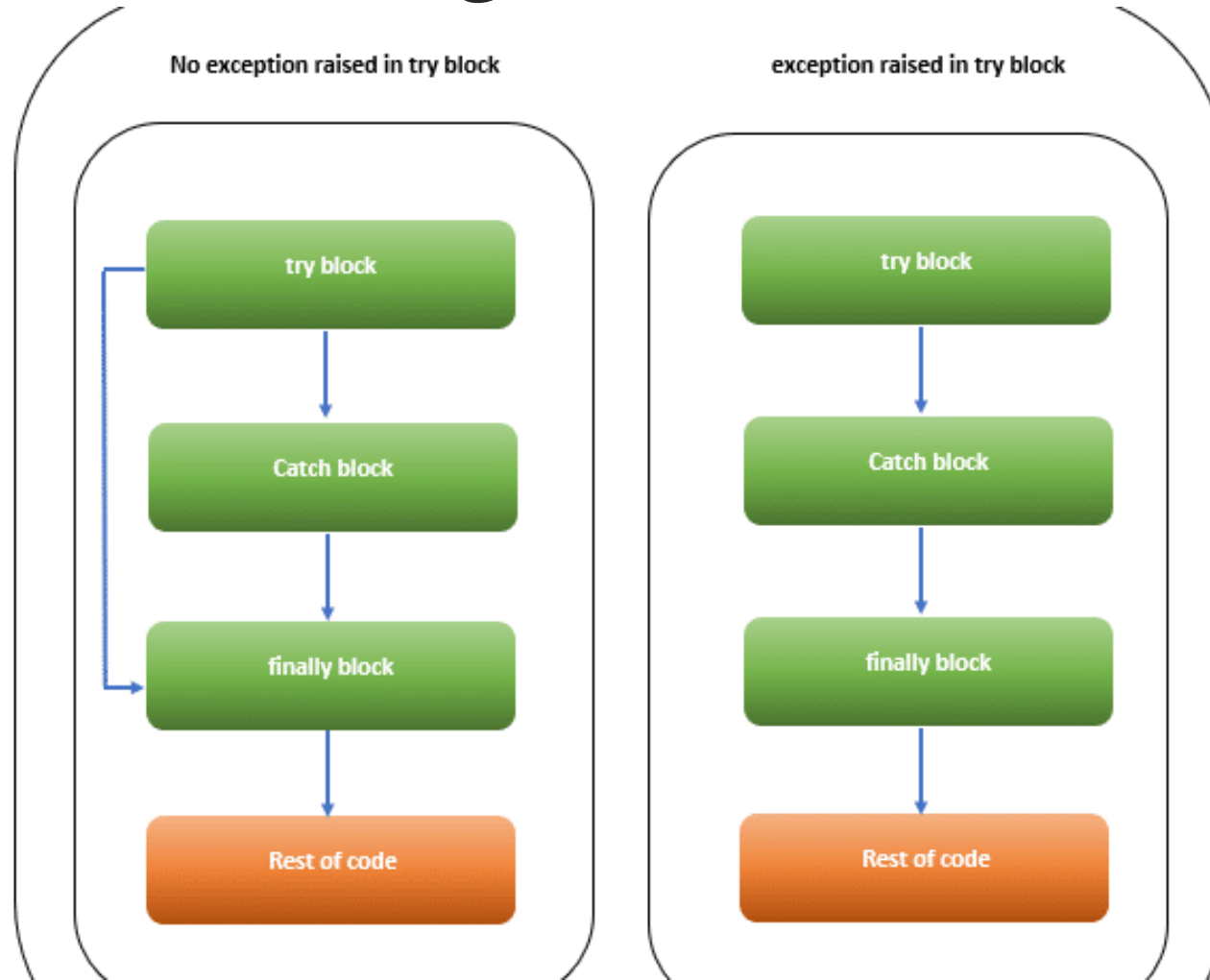
run:

This block will always execute

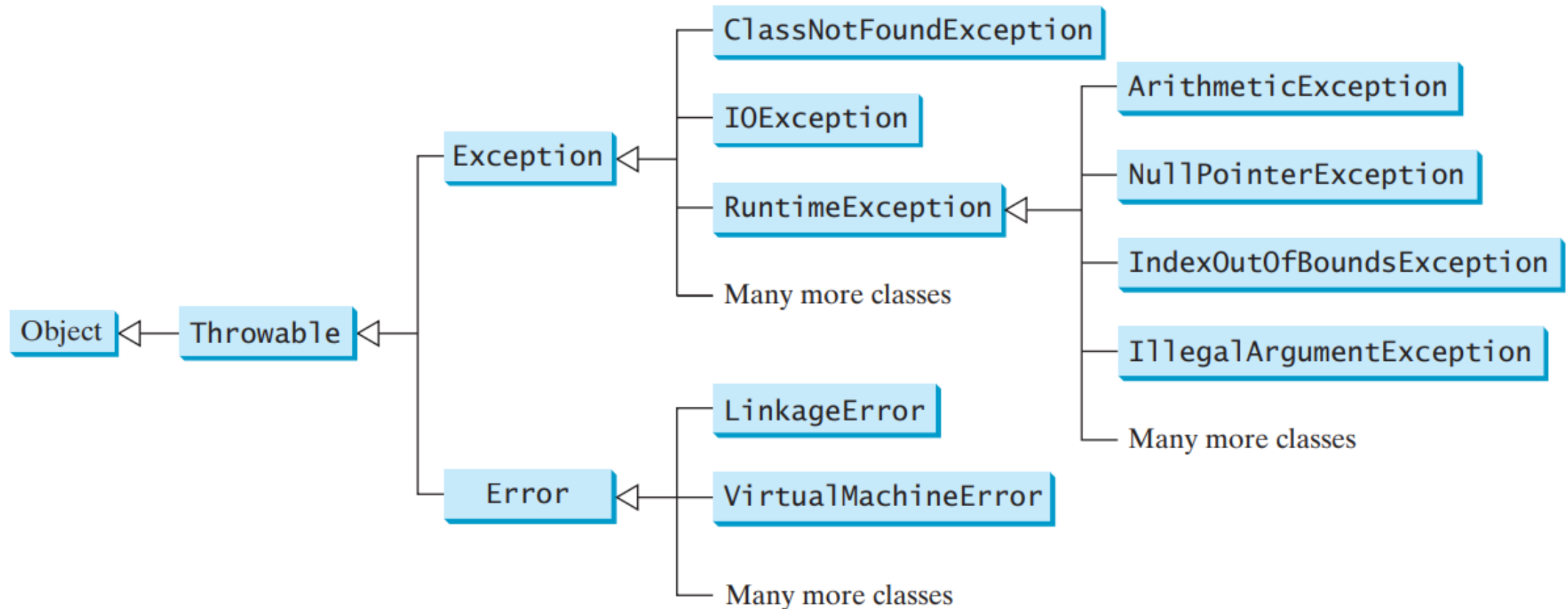
Rest of the code after try-finally block

BUILD SUCCESSFUL (total time: 0 seconds)

Exception Handling



Exception Handling



Exception Handling – Throw and Throws

- **Throw:**

- Purpose: Used to explicitly throw an exception from within a method or any block of code
- You use throw when you want to manually throw an exception at a specific point in the code

- **Throws:**

- Declares that a method might throw one or more exceptions. It serves as a warning for the caller of the method to handle those exceptions

Exception Handling – Throws and Throw

```
public class SENG201_Week_8 {  
    public static void main(String[] args) {  
        // TODO code application logic here  
        divide(14,0);  
        System.out.println("Continue with the rest of the main");  
    }  
    public static void divide(int a, int b) {  
        try {  
            int result = a / b;  
            System.out.println("Result: " + result);  
        } catch (Exception e) {  
            System.out.println("Logging: Cannot divide by zero.");  
            //throw e; // Re-throw the exception to be handled elsewhere  
        }  
    }  
}
```

Use **throw** if you want to re-throw an exception after handling it.

```
public static void main(String[] args) {  
    // TODO code application logic here  
    try{  
        divide(14,0);  
    }  
    catch (Exception e){  
        System.out.println("Exception " + e);  
    }  
    finally{  
        System.out.println("In main finally");  
    }  
    System.out.println("Continue with the main");  
}  
  
public static void divide(int a, int b) {  
    try {  
        int result = a / b;  
        System.out.println("Result: " + result);  
    } catch (Exception e) {  
        System.out.println("Logging: Cannot divide by zero.");  
        throw e; // Re-throw the exception to be handled elsewhere  
    }  
    finally  
    {  
        System.out.println("In function finally");  
    }  
}
```

Exception Handling – Throws and Throw

```
public class SENG201_Week_8 {  
    public static void main(String[] args) {  
        // TODO code application logic here  
        try{  
            divide(14,0);  
        }  
        catch (Exception e){  
            System.out.println("Exception " + e);  
        }  
        System.out.println("Continue with the main");  
    }  
    public static int divide(int a, int b) throws Exception {  
        if (b == 0) {  
            throw new Exception("Cannot divide by zero.");  
        }  
        return a / b;  
    }  
}
```

- >Throws is added to the method signature to indicate that this method may throw an exception
- >Allows the caller to handle the exception, making it clear that the caller needs to take care of potential errors
- >Use Throws if you do not handle the exception and want the caller to be responsible for it

Exception Handling – Getting Information

java.lang.Throwable

```
+getMessage(): String  
+toString(): String  
  
+printStackTrace(): void  
  
+getStackTrace():  
  StackTraceElement[]
```

Returns the message that describes this exception object.

Returns the concatenation of three strings: (1) the full name of the exception class; (2) ":" (a colon and a space); (3) the `getMessage()` method.

Prints the `Throwable` object and its call stack trace information on the console.

Returns an array of stack trace elements representing the stack trace pertaining to this exception object.

Exception Handling – Getting Information



```
public class SENG201_Week_8 {
    public static void main(String[] args) {
        // TODO code application logic here
        try{
            divide(14,0);
        }
        catch (Exception e){
            System.out.println(e.getMessage());
            System.out.println(e.toString());
            e.printStackTrace();
            StackTraceElement[] traceElements = e.getStackTrace();
            for (int i = 0; i < traceElements.length; i++) {
                System.out.print("method " + traceElements[i].getMethodName());
                System.out.print("(" + traceElements[i].getClassName() + ":");
                System.out.println(traceElements[i].getLineNumber() + ")");
            }
        }
    }

    public static int divide(int a, int b) throws Exception {
        if (b == 0) {
            throw new Exception("Cannot divide by zero.");
        }
        return a / b;
    }
}
```

run:

Cannot divide by zero.

java.lang.Exception: Cannot divide by zero.

java.lang.Exception: Cannot divide by zero.

at seng201_week_8.SENG201_Week_8.divide(SENG201_Week_8.java:32)

at seng201_week_8.SENG201_Week_8.main(SENG201_Week_8.java:16)

method divide(seng201_week_8.SENG201_Week_8:32)

method main(seng201_week_8.SENG201_Week_8:16)

BUILD SUCCESSFUL (total time: 0 seconds)

SENG 201 – Object Oriented Programming

2024 – 2025 Fall Lectures – Week 10

Ömer Mintemur



List of Objects

```
public class Person {  
  
    private String name;  
    private String surname;  
    private int age;  
  
    Person() {  
    }  
  
    Person(String name_p, String surname_p, int age_p)  
    {  
        name = name_p;  
        surname = surname_p;  
        age = age_p;  
    }  
    public void showInfo()  
    {  
        System.out.println("Name is " + name +  
            " Surname " + surname + " Age " + age);  
    }  
}
```

```
public static void main(String[] args) {  
    // TODO code application logic here  
    Person[] persons = new Person[10];  
    persons[0].showInfo();  
}
```

Null Pointer Exception

This creates an array of `Person` references, but the array elements are initially set to `null`. There are no actual `Person` objects in the array at this point—just `null` references.

List of Objects

```
public class Person {  
  
    private String name;  
    private String surname;  
    private int age;  
  
    Person() {  
  
    }  
  
    Person(String name_p, String surname_p, int age_p)  
    {  
        name = name_p;  
        surname = surname_p;  
        age = age_p;  
    }  
    public void showInfo()  
    {  
        System.out.println("Name is " + name +  
            " Surname " + surname + " Age " + age);  
    }  
}
```

WRONG !!

```
public static void main(String[] args) {  
    // TODO code application logic here  
    Person[] persons = new Person[10];  
    persons[0].showInfo();  
}
```

SOLUTION

```
public static void main(String[] args) {  
    // TODO code application logic here  
    Person[] persons = new Person[10];  
    persons[0] = new Person("John", "Sun", 20);  
    persons[0].showInfo();  
}
```

```
public static void main(String[] args) {  
    // TODO code application logic here  
    Person[] persons = {new Person(), new Person("John", "Sun", 20)};  
}
```


Static in Class

```
public class Person {  
  
    private String name;  
    private String surname;  
    private int age;  
  
    private static int object_count = 0;  
  
    Person() {  
        object_count++;  
    }  
  
    Person(String name_p, String surname_p, int age_p)  
    {  
        name = name_p;  
        surname = surname_p;  
        age = age_p;  
        object_count++;  
    }  
  
    public void showInfo()  
    {  
        System.out.println("Name is " + name +  
            " Surname " + surname + " Age " + age);  
    }  
  
    public static int getObjectCount()  
    {  
        return object_count;  
    }  
}
```

```
public static void main(String[] args) {  
    // TODO code application logic here  
    Person p1 = new Person();  
    Person p2 = new Person();  
    Person p3 = new Person();  
    System.out.println(p1.getObjectCount());  
    System.out.println(p2.getObjectCount());  
    System.out.println(p3.getObjectCount());  
    System.out.println(Person.getObjectCount());  
}
```

Static Variable:

- A static variable belongs to the class, not to any specific object
- All instances of the class share the same static variable

Static Methods:

- A static method can access and modify static variables
- Static methods cannot access instance variables or instance methods directly because they do not have a reference to a specific object.

This Keyword

```
public class Cat {  
  
    private String name;  
    private int age;  
    private String type;  
  
    // Default Constructor  
    Cat()  
    {  
  
    }  
    Cat (String name, int age, String type)  
    {  
        name = name;  
        age = age;  
        type = type;  
    }  
  
    public void show()  
    {  
        System.out.println("Cat name is " + name +  
            "\nAge is " + age + "\nType is " + type);  
    }  
}
```

```
public static void main(String[] args) {  
    // TODO code application logic here  
    Cat cat_1 = new Cat("John", 2, "Scottish");  
    cat_1.show();  
}
```

run:

Cat name is null

Age is 0

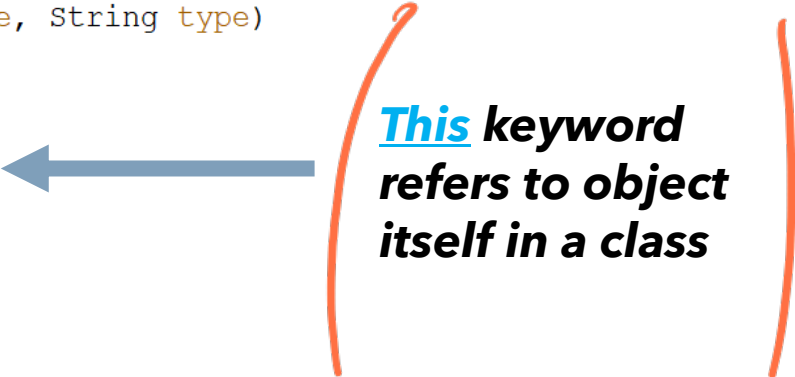
Type is null

BUILD SUCCESSFUL

This Keyword

```
public class Cat {  
  
    private String name;  
    private int age;  
    private String type;  
  
    // Default Constructor  
    Cat()  
    {  
  
    }  
  
    Cat (String name, int age, String type)  
    {  
        this.name = name;  
        this.age = age;  
        this.type = type;  
    }  
  
    public void show()  
    {  
        System.out.println("Cat name is " + name +  
            "\nAge is " + age + "\nType is " + type);  
    }  
}
```

**This keyword
refers to object
itself in a class**



```
public static void main(String[] args) {  
    // TODO code application logic here  
    Cat cat_1 = new Cat("John", 2, "Scottish");  
    cat_1.show();  
}
```

```
run:  
Cat name is John  
Age is 2  
Type is Scottish  
BUILD SUCCESSFUL
```

This Keyword

- **This** keyword refers to the current object in a method or constructor. It allows access to the instance variables and the methods of the class
- When a constructor or method has parameters with the same name as instance variables, the **this** keyword is used to distinguish the instance variables from the parameters.

https://search.app?link=https%3A%2F%2Fchatgpt.com%2Fc%2F6746d631-91e4-8005-bd0d-9571471cdceb&utm_campaign=aga&utm_source=agsadl2%2Csh%2Fx%2Fgs%2Fm2%2F4

```
public class Person {  
    private String name;  
  
    // Constructor  
    public Person(String name) {  
        this.name = name;  
    }  
}
```

```
// Mevcut nesneyi bir sınıfa gönderen metod  
public void printPerson(PersonPrinter printer) {  
    printer.print(this); // Mevcut nesneyi gönderiyoruz.  
}
```

```
class PersonPrinter {  
    public void print(Person person) {  
        System.out.println("Person's name: " + person.name);  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Person person = new Person("John"); // Bir nesne oluşturduk  
        PersonPrinter printer = new PersonPrinter(); // Yazdırıcı sınıf nesnesi oluşturuldu  
  
        person.printPerson(printer); // Mevcut nesneyi gönderiyoruz  
    }  
}
```

← mevcut olan nesneyi bir sınıfa gösterebilmek için nesnenin içinde bulunan fonksiyonu kullanmak gerekir.

This Keyword – Compare Two Cats

```
public static void main(String[] args) {  
    // TODO code application logic here  
    Cat cat_1 = new Cat("John", 2, "Scottish");  
    Cat cat_2 = new Cat("Mike", 1, "British");  
  
    // Is the type of cat_1 is equal to cat_2  
  
    if (cat_1.getType()==cat_2.getType())  
    {  
        System.out.println("Cats' type are equal");  
    }  
    else  
    {  
        System.out.println("Cats' type are not equal");  
    }  
}
```

```
public class Cat {  
  
    private String name;  
    private int age;  
    private String type;  
    // Default Constructor  
    Cat(){}  
    Cat (String name, int age, String type)  
    {  
        this.name = name;  
        this.age = age;  
        this.type=type;  
    }  
    public void show()  
    {  
        System.out.println("Cat name is " + name +  
        "\nAge is " + age + "\nType is " + type);  
    }  
    public String getType()  
    {  
        return this.type;  
    }  
}
```

This Keyword – Compare Two Cats (V2)

```
public static void main(String[] args) {  
    // TODO code application logic here  
    Cat cat_1 = new Cat("John", 2, "Scottish");  
    Cat cat_2 = new Cat("Mike", 1, "British");  
  
    // Is the type of cat_1 is equal to cat_2  
    cat_1.compareCat(cat_2);  
  
}  
  
// Is the type of cat_1 is equal to cat_2  
//cat_1.compareCat(cat_2);  
cat_2.compareCat(cat_1);
```

```
public class Cat {  
  
    private String name;  
    private int age;  
    private String type;  
    // Default Constructor  
    Cat() {}  
    Cat (String name, int age, String type)  
    {  
        this.name = name;  
        this.age = age;  
        this.type=type;  
    }  
    public void show()  
    {  
        System.out.println("Cat name is " + name +  
            "\nAge is " + age + "\nType is " + type);  
    }  
  
    public void compareCat(Cat c)  
    {  
        if(this.type == c.getType())  
        {  
            System.out.println("Cats' type are equal");  
        }  
        else  
        {  
            System.out.println("Cats' type are not equal");  
        }  
    }  
  
    public String getType()  
    {  
    }  
}
```

This Keyword

```
public static void main(String[] args) {  
    // TODO code application logic here  
    Cat cat_1 = new Cat("John", 2, "Scottish");  
    Cat cat_2 = new Cat("Mike", 1, "British");  
  
    Cat cat_3 = cat_1.returnCat();  
    cat_3.show();  
}
```

```
public class Cat {  
  
    private String name;  
    private int age;  
    private String type;  
    // Default Constructor  
    Cat() {}  
    Cat (String name, int age, String type)  
    {  
        this.name = name;  
        this.age = age;  
        this.type = type;  
    }  
    public void show()  
    { ...4 lines }  
    public void compareCat(Cat c)  
    { ...10 lines }  
    public Cat returnCat()  
    {  
        return this;  
    }  
    public String getType()  
    { ...3 lines }
```

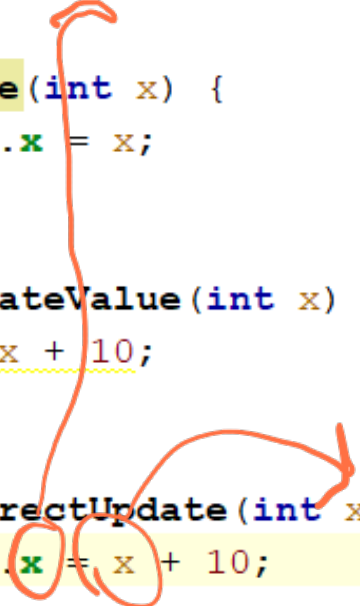
This Keyword

```
public class Trick {  
    private int x;  
    private int y;  
  
    // Constructor  
    Trick(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    // Method to update 'x'  
    Trick setX(int x) {  
        this.x = x;  
        return this;  
    }  
  
    // Method to update 'y'  
    Trick setY(int y) {  
        this.y = y;  
        return this;  
    }  
  
    // Method to display the values  
    void display() {  
        System.out.println("x = " + this.x + ", y = " + this.y);  
    }  
}
```

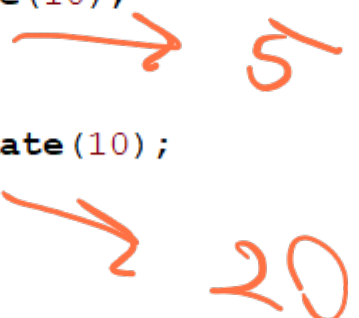
```
public static void main(String[] args) {  
    // TODO code application logic here  
    Trick obj = new Trick(1, 2);  
  
    obj.setX(10).setY(20).display();  
  
    Trick obj2 = new Trick(5, 5);  
    obj2.display();  
}
```


This Keyword

```
public class TrickMore {  
    private int x;  
  
    TrickMore(int x) {  
        this.x = x;  
    }  
  
    void updateValue(int x) {  
        x = x + 10;  
    }  
  
    void correctUpdate(int x) {  
        this.x = x + 10;  
    }  
  
    void display() {  
        System.out.println("x = " + this.x);  
    }  
}
```



```
public static void main(String[] args) {  
    // TODO code application logic here  
    TrickMore obj = new TrickMore(5);  
  
    obj.updateValue(10);  
    obj.display();  
  
    obj.correctUpdate(10);  
    obj.display();  
}
```



Inheritance

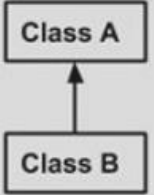
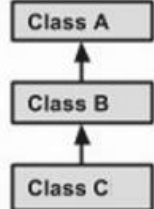
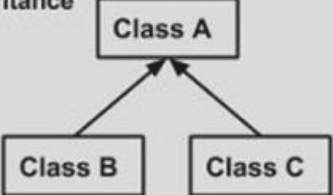
- What is Inheritance?
 - Inheritance is a mechanism in OOP where a class (child/subclass) derives properties and behaviors (attributes and methods) from another class (parent/superclass)
- Enable code reusability, modularity and hierarchy representation.

Inheritance

- How it works?
 - The extends keyword is used to establish inheritance
 - A subclass inherits all **public** members of its parent class
- **Private** members of the parent are not directly accessible but can be accessed through getters/setters or protected methods

Inheritance

- **Single Inheritance:** A class inherits from one parent class.
- **Multilevel Inheritance:** A class inherits from a class, which itself inherits from another class
- **Hierarchical Inheritance:** Multiple subclasses inherit from a single parent class
- **Multiple Inheritance is not available in Java**

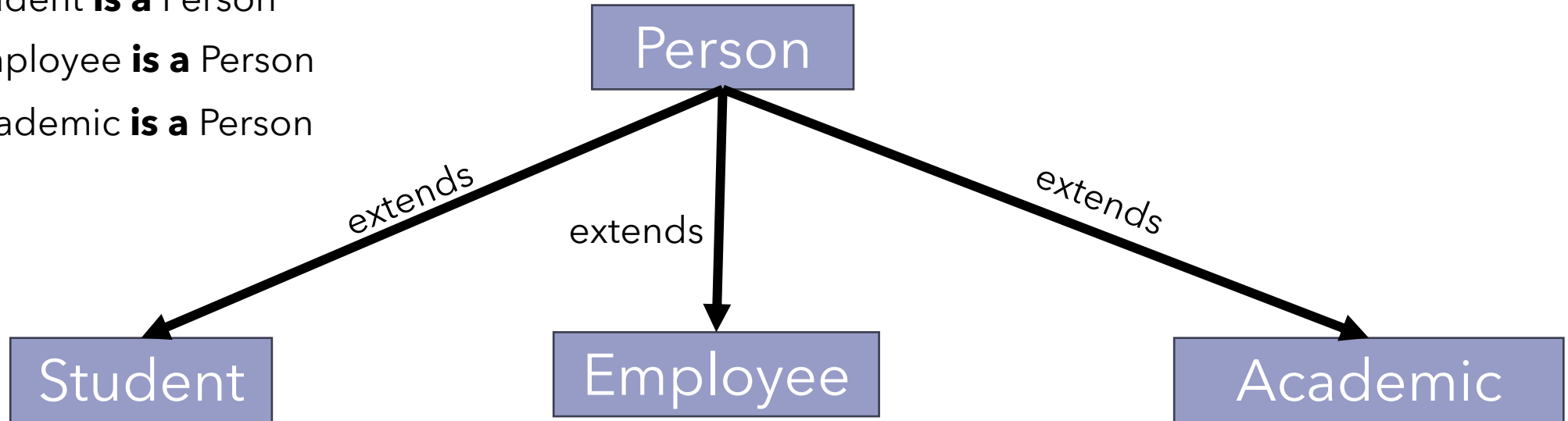
Single Inheritance  <pre>graph BT; B[Class B] --> A[Class A]</pre>	<pre>public class A { } public class B extends A { }</pre>
Multi Level Inheritance  <pre>graph BT; C[Class C] --> B[Class B]; B --> A[Class A]</pre>	<pre>public class A {} public class B extends A {.....} public class C extends B {.....}</pre>
Hierarchical Inheritance  <pre>graph BT; B[Class B] --> A[Class A]; C[Class C] --> A</pre>	<pre>public class A {} public class B extends A {.....} public class C extends A {.....}</pre>

Inheritance

Each Student **is a** Person

Each Employee **is a** Person

Each Academic **is a** Person



*Use inheritance when there is a clear **"is-a" relationship** between classes*

Inheritance

```
public class Person {  
    Person() {}  
  
    public void sayHi()  
    {  
        System.out.println("I am a Person");  
    }  
}
```

```
public static void main(String[] args) {  
    // TODO code application logic here  
    Person p1 = new Person();  
    p1.sayHi();  
    Student s1 = new Student();  
    s1.sayHi();  
}
```

```
public class Student extends Person{  
  
    Student() {}  
  
}
```

```
run:  
I am a Person  
I am a Person
```

Inheritance - Polymorphism

```
public class Person {  
    Person() {}  
  
    public void sayHi()  
    {  
        System.out.println("I am a Person");  
    }  
}
```

```
public class Student extends Person{  
  
    Student() {}  
  
    public void sayHi()  
    {  
        System.out.println("I am a Student");  
    }  
}
```

```
public static void main(String[] args) {  
    // TODO code application logic here  
    Person p1 = new Person();  
    p1.sayHi();  
    Student s1 = new Student();  
    s1.sayHi();  
}
```

```
run:  
I am a Person  
I am a Student
```

*A subclass provides a **specific implementation of a method defined in its parent class. If not overridden, all public methods in extended class can be used in subclass.***

Inheritance

```
public class Person {  
    public String name;  
    public String surname;  
    public int age;  
    Person() {}  
  
    public void sayHi()  
    {  
        System.out.println("I am a Person");  
    }  
  
    public class Student extends Person{  
        Student() {}  
  
        public void sayHi()  
        {  
            System.out.println("I am a Student");  
        }  
    }  
}
```

```
public static void main(String[] args) {  
    // TODO code application logic here  
    Student s1 = new Student();  
    s1.  
}
```

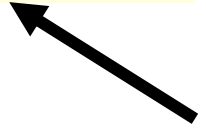
age	int
name	String
surname	String
equals (Object obj)	boolean
getClass ()	Class<?>
hashCode ()	int
notify()	void
notifyAll()	void
sayHi ()	void
toString()	String
wait()	void
wait(long timeoutMillis)	void
wait(long timeoutMillis, int nanos)	void

Inheritance

```
public class Person {  
    private String name;  
    private String surname;  
    private int age;  
    Person() {}  
  
    Person(String name, String surname, int age)  
    {  
        this.name = name;  
        this.surname = surname;  
        this.age = age;  
    }  
  
    public void sayHi()  
    {  
        System.out.println("I am a Person");  
    }  
}
```

```
public class Student extends Person{  
  
    private int studentId;  
  
    Student() {}  
    Student(int studentId)  
    {  
        this.studentId = studentId;  
    }  
  
    public void sayHi()  
    {  
        System.out.println("I am a Student");  
    }  
  
    public static void main(String[] args) {  
        // TODO code application logic here  
        Student s1 = new Student(123);  
    }  
}
```

Since every student is a Person, each Student should have name, surname, and age. Moreover, every student should have student id.



Inheritance

```
public class Person {  
    private String name;  
    private String surname;  
    private int age;  
    Person() {}  
  
    Person(String name, String surname, int age)  
    {  
        this.name = name;  
        this.surname = surname;  
        this.age = age;  
    }  
    public void sayHi()  
    {  
        System.out.println("I am a Person");  
    }  
    public String showInfo()  
    {  
        return "Name is " + this.name + " Surname is "  
        + this.surname + " Age is " + this.age;  
    }  
}
```

```
public class Student extends Person{  
    private int studentId;  
    Student() {}  
    Student(String name, String surname, int age, int studentId)  
    {  
        super(name, surname, age);  
        this.studentId = studentId;  
    }  
    public void sayHi()  
    {  
        System.out.println("I am a Student");  
    }  
    public String showInfo()  
    {  
        return super.showInfo() +  
        " Student ID is " + this.studentId;  
    }  
}
```

Inheritance (Not extendable)

```
public final class Person {  
    private String name;  
    private String surname;  
    private int age;  
    Person() {}  
  
    Person(String name, String surname, int age)  
    {  
        this.name = name;  
        this.surname = surname;  
        this.age = age;  
    }  
}
```

You can not extend a class if it is defined as final

```
public class Person {  
    private String name;  
    private String surname;  
    private int age;  
    Person() {}  
  
    Person(String name, String surname, int age)  
    {  
        this.name = name;  
        this.surname = surname;  
        this.age = age;  
    }  
    public final void sayHi()  
    {  
        System.out.println("I am a Person");  
    }  
    public String showInfo()  
    {  
        return "Name is " + this.name + " Surname is "  
            + this.surname + " Age is " + this.age;  
    }  
}
```

You can not override a method if it is defined as final

Inheritance (Ordering)

```
public class Person {  
    Person() {  
        System.out.println("Person constructor");  
    }  
}
```

```
public class JuniorStudent extends Student {  
    JuniorStudent()  
    {  
        System.out.println("Junior Student Constructor");  
    }  
}
```

```
public class Student extends Person{  
    Student() {  
        System.out.println("Student Constructor");  
    }  
  
    public static void main(String[] args) {  
        // TODO code application logic here  
        JuniorStudent j1 = new JuniorStudent();  
    }  
}
```

run:
Person constructor
Student Constructor
Junior Student Constructor

QUESTIONS

- 1) Average acceleration is defined as the change of velocity divided by the time taken to make the change, as shown in the following formula

$$a = \frac{v_1 - v_0}{t}$$

Write a program that prompts the user to enter the starting velocity v_0 , the ending velocity v_1 , and the time span t , and displays the average acceleration (40).

Here is a simple run:

```
run:
Enter v0, v1 and t: 5,5 50,9 4,5
The average acceleration is 10.088889
BUILD SUCCESSFUL (total time: 10 seconds)
```

- 2) Write a program that prompts the user to enter an integer number of seconds and displays the equivalent number of hours and remaining minutes and seconds (60).

```
run:
Enter the number of seconds: 1000000
1000000 seconds is equal to 277 hours, 46 minutes, and 40 seconds.
BUILD SUCCESSFUL (total time: 3 seconds)
```

SENG 201 – Object Oriented Programming

2024 – 2025 Fall Lectures – Week 11

Ömer Mintemur



Reference Type vs Object Type

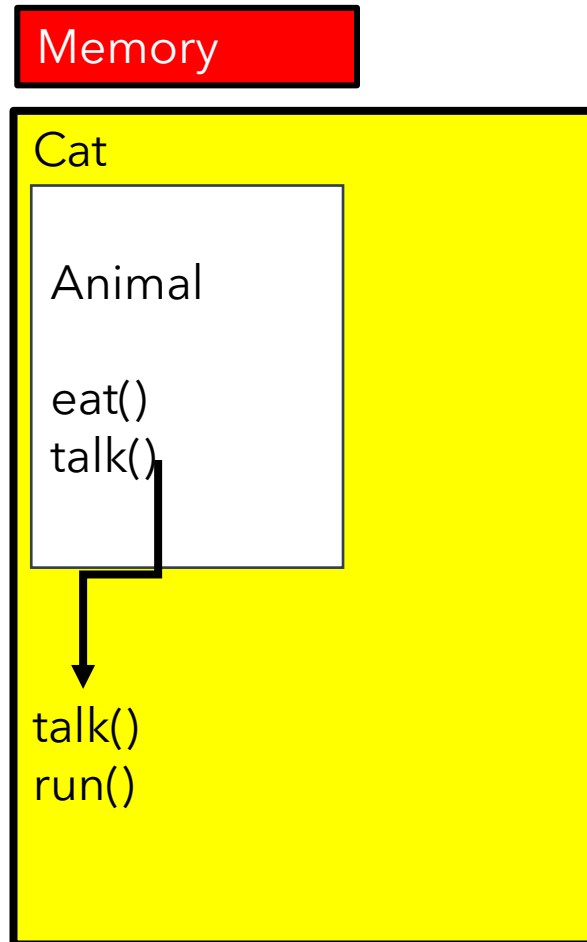
```
class Animal{  
    Animal() {}  
    public void talk() {  
        System.out.println("I am an Animal");  
    }  
}  
class Cat extends Animal{  
    Cat() {}  
    @Override  
    public void talk()  
    {  
        System.out.println("I am a Cat");  
    }  
}  
public class Seng201_Week_11 {  
    public static void main(String[] args) {  
        Animal a = new Cat();  
    }  
}
```

Ensures that the overridden method matches a method in the superclass. If there's a mistake (e.g., method signature mismatch or wrong name), the compiler will generate an error.

Reference Type

Object Type

Reference Type vs Object Type



```
Animal a = new Cat()  
a.talk()  
a.run()  
a.eat()
```


Reference Type vs Object Type

Aspect	Reference Type	Object Type
<i>Definition</i>	The type of the variable holding the object reference.	The actual class of the object created in memory.
<i>What It Controls</i>	What methods or fields can be accessed from the reference.	The behavior of the object (polymorphism).
<i>Example</i>	Animal a in Animal a = new Cat()	Cat in new Cat()

```
public static void main(String[] args) {  
    Animal a = new Cat();  
    a.talk();  
}
```

Output

run:

I am a Cat

- **Reference type determines *what methods are accessible* at compile time**
- **Object type determines *what method implementation* is executed at runtime**

Reference Type vs Object Type

```
class Animal{
    Animal() {}
    public void talk() {
        System.out.println("I am an Animal");
    }
}

class Cat extends Animal{
    Cat() {}
    @Override
    public void talk()
    {
        System.out.println("I am a Cat");
    }
    public void run()
    {
        System.out.println("I am running");
    }
}
```

```
public static void main(String[] args) {
    Animal a = new Cat();
    a.talk();
    a.run();
}
```

- **Why can not access run()?**
- **Polymorphism Rule:** You can only access the methods that are defined in the reference type (**Animal** in this case), even if the actual object type (**Cat**) has additional methods.
- run() is not defined in **Animal**, so it cannot be accessed using an **Animal** reference.

Reference Type vs Object Type

```
class Animal{
    Animal() {}
    public void talk() {
        System.out.println("I am an Animal");
    }
}

class Cat extends Animal{
    Cat() {}
    @Override
    public void talk()
    {
        System.out.println("I am a Cat");
    }
    public void run()
    {
        System.out.println("I am running");
    }
}
```

Solution

```
public static void main(String[] args) {
    Animal a = new Cat();
    a.talk();
    Cat c = (Cat)a;
    c.run();
}
```

UPCASTING

DOWNCASTING

Upcasting & Downcasting

UPCASTING

- **Definition:** Converting a subclass object reference to a superclass type
- **Implicit:** Happens automatically in Java without the need for explicit casting
- **Limitation:** The reference can only access methods and fields defined in the superclass, not the subclass-specific ones

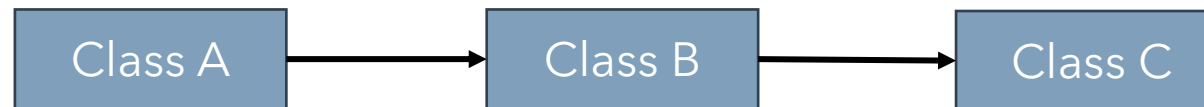
DOWNCASTING

- **Definition:** Converting a superclass reference back to a subclass type
- **Explicit:** Requires explicit casting using the syntax
- **Risk:** If the reference doesn't actually point to an object of the target subclass, it will throw a ***ClassCastException*** at runtime

```
Animal a = new Cat();
```

```
Cat c = (Cat) a;
```

Downcasting



Upcasting

Risk of Downcasting

```
class Animal{
| Animal() {}
| public void talk() {
|     System.out.println("I am an Animal");}
| }
class Cat extends Animal{
| Cat() {}
| @Override
| public void talk() {
|     System.out.println("I am a Cat");}
| }
class Dog extends Animal {
|     Dog() {}
|     public void talk() {
|         System.out.println("I am a Dog");}
| }
```

Animal a = new Cat();

Cat c = (Cat) a;

Dog d = (Dog) a;

Exception in thread "main" java.lang.ClassCastException:

Risk of Downcasting – Use *instanceof*

```
Animal a = new Cat();  
if (a instanceof Animal)  
{  
    System.out.println("a is instanceof Animal");  
}  
if(a instanceof Cat)  
{  
    System.out.println("a is instance of Cat");  
}  
if(a instanceof Dog)  
{  
    System.out.println("a is instance of Dog");  
}
```

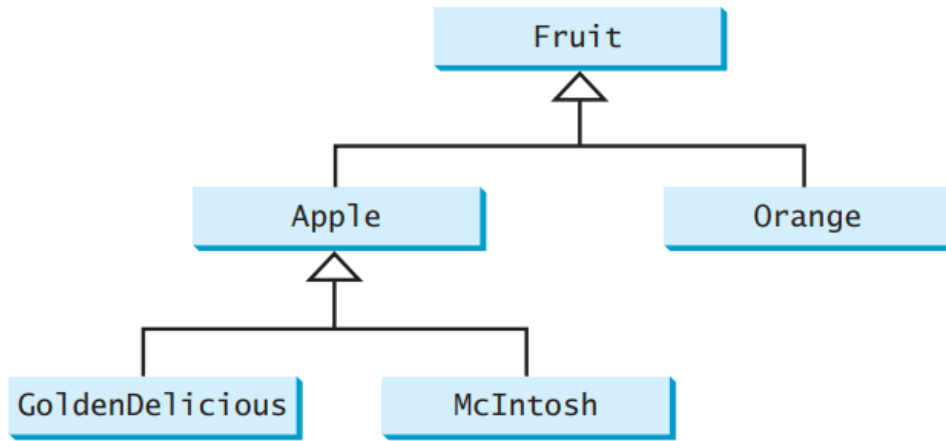
run:

```
a is instanceof Animal  
a is instance of Cat
```

Purpose of *instanceof*

- Type Checking
 - Before attempting to cast an object to a specific subclass, *instanceof* ensures that the object is indeed the expected type. This prevents `ClassCastException` at runtime
- Polymorphism Handling
 - When working with a superclass reference that points to a subclass object, *instanceof* helps determine the actual type of the object to execute type-specific behaviors
- Dynamic Type Identification
 - Useful in scenarios where a collection or method deals with objects of multiple types

Purpose of *instanceof*



Assume that the following code is given:

```
Fruit fruit = new GoldenDelicious();
Orange orange = new Orange();
```

```
System.out.println(fruit.getClass());
System.out.println(fruit.getClass().getSuperclass());
System.out.println(fruit.getClass().getSuperclass().getSuperclass());
System.out.println(fruit.getClass().getSuperclass().getSuperclass().getSuperclass());
System.out.println(fruit.getClass().getSuperclass().getSuperclass().getSuperclass().getSuperclass());
```

Is `fruit instanceof Fruit`?

Is `fruit instanceof Orange`?

Is `fruit instanceof Apple`?

Is `fruit instanceof GoldenDelicious`?

Is `fruit instanceof McIntosh`?

Is `orange instanceof Orange`?

run:

```
class Package_Fruit.GoldenDelicious
class Package_Fruit.Apple
class Package_Fruit.Fruit
class java.lang.Object
```


Packages

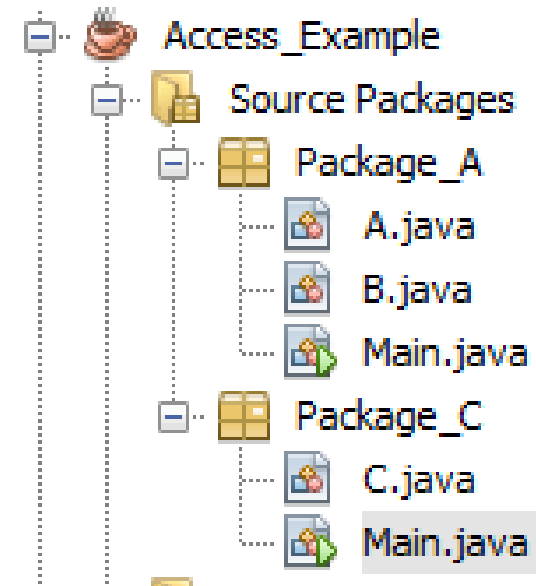
A **package** is a namespace that organizes Java classes into groups

Access Control: Classes in a package can have default access, making them accessible only within the package

Reusability: Promotes modularity and reusability of code

Types of Packages:

- **Built-in Packages:** Provided by Java
- **User-defined Packages:** Created by developers to organize their own code



Package Example

```
package packageexample;
```

```
import java.lang.Math.*;
```

```
public class PackageExample {
```

```
    public static void main(String[] args) {
```

```
        int x = 2;
```

```
        double x_square = Math.pow(x, 2);
```

```
    }
```

```
}
```

static double	abs (double a) Returns the absolute value of a double value.
---------------	--

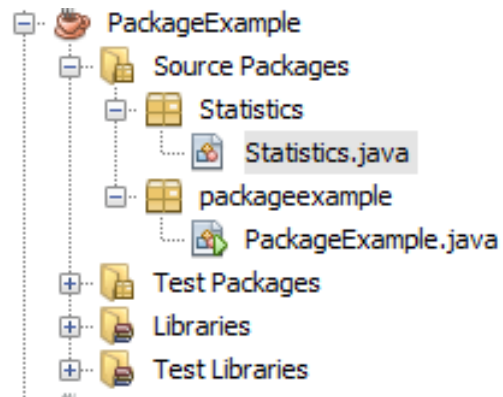
static float	abs (float a) Returns the absolute value of a float value.
--------------	--

static int	abs (int a) Returns the absolute value of an int value.
------------	---

static long	abs (long a) Returns the absolute value of a long value.
-------------	--

static double	acos (double a) Returns the arc cosine of a value; the returned angle is in the range 0.0 through <i>pi</i> .
---------------	---

Package Example



```
package packageexample;
import Statistics.*;

public class PackageExample {

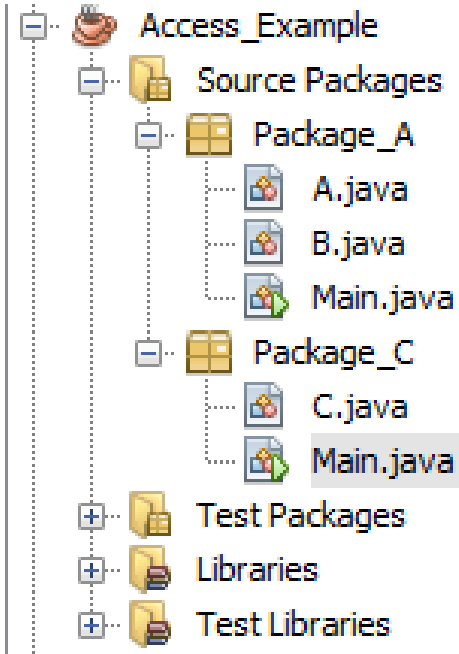
    public static void main(String[] args) {

        int[] my_array = {1,2,3,4,5,6,7};
        double avr = Statistics.average(my_array);
        System.out.println(avr);
        double sd = Statistics.standardDeviation(my_array);
        System.out.println(sd);
    }
}
```

```
public class Statistics {
    public static double average(int[] array) throws IllegalArgumentException
    {
        if (array.length == 0) {
            throw new IllegalArgumentException("Array cannot be empty");
        }
        int sum = 0;
        for (int num : array) {
            sum += num;
        }
        return (double) sum / array.length;
    }

    public static double standardDeviation(int[] array) throws IllegalArgumentException
    {
        if (array.length == 0) {
            throw new IllegalArgumentException("Array cannot be empty");
        }
        double sum = 0.0;
        for (double num : array) {
            sum += num;
        }
        double mean = sum / array.length;
        double squaredDifferenceSum = 0.0;
        for (double num : array) {
            squaredDifferenceSum += Math.pow(num - mean, 2);
        }
        double variance = squaredDifferenceSum / array.length;
        return Math.sqrt(variance);
    }
}
```

Package Accessibility(Which Modifier Can Access Which?)



In Package_A:

- We have Class A
- We have Class B
- We have a Main

In Package_C:

- We have Class C
- We have a Main

Package Accessibility(Which Modifier Can Access Which?)

```
public class A {  
    private int private_integer_A;  
    public int public_integer_A;  
    protected int protected_integer_A;  
    A() {}  
}
```

```
public class B extends A {  
    private int private_integer_B;  
    public int public_integer_B;  
    protected int protected_integer_B;  
    B() {}  
}
```

```
public class B extends A {  
    private int private_integer_B;  
    public int public_integer_B;  
    protected int protected_integer_B;  
    B() {  
        int dummy = this.protected_integer_A;  
        int dummy_2 = this.public_integer_A;  
    }  
}
```

B can access public and protected variables/methods in **A**.

Package Accessibility(Which Class Can Access Which Modifier?)

Package A

```
public class A {  
    private int private_integer_A;  
    public int public_integer_A;  
    protected int protected_integer_A;  
    A() {}  
}
```

Default (means no modifier) can only be accessed from the same package

Package C

```
public class C extends A{  
    private int private_integer_C;  
    public int public_integer_C;  
    protected int protected_integer_C;  
    C(){  
        int a = this.protected_integer_A;  
    }  
}
```

A() is not public in package A; cannot be accessed from outside package

Package Accessibility(Which Class Can Access Which Modifier?)

Package A

```
public class A {  
    private int private_integer_A;  
    public int public_integer_A;  
    protected int protected_integer_A;  
    A() {}  
}
```

```
public class B extends A {  
    private int private_integer_B;  
    public int public_integer_B;  
    protected int protected_integer_B;  
    B() {  
        int dummy = this.protected_integer_A;  
        //super.protected_integer_A;  
        int dummy_2 = this.public_integer_A;  
        //super.public_integer_A;  
    }  
}
```

Default modifiers can successfully be accessed by the class in same package.

Package Accessibility(Which Class Can Access Which Modifier?)

Package A

```
public class A {  
    private int private_integer_A;  
    public int public_integer_A;  
    protected int protected_integer_A;  
    A() {}  
}
```

Package C

```
public class C extends A{  
    private int private_integer_C;  
    public int public_integer_C;  
    protected int protected_integer_C;  
    C() {  
        int a = this.protected_integer_A;  
    }  
}
```

```
public class A {  
    private int private_integer_A;  
    public int public_integer_A;  
    protected int protected_integer_A;  
    public A() {}  
    //protected A() {}  
}
```

```
public class C extends A{  
    private int private_integer_C;  
    public int public_integer_C;  
    protected int protected_integer_C;  
    C() {  
        int dummy = this.protected_integer_A;  
        //super.protected_integer_A;  
        int dummy_2 = this.public_integer_A;  
        //super.public_integer_A;  
    }  
}
```


Package Accessibility(Which Class Can Access Which Modifier?)

Modifier	Within Same Package	Outside of the Package
<i>public</i>	Accessible everywhere	Accessible everywhere
<i>protected</i>	Accessible	Accessible only through inheritance
<i>default</i>	Accessible	Not accessible
<i>private</i>	Not accessible	Not accessible

- **public**: Open to all classes in all packages
- **protected**: Limited to the same package and subclasses outside the package
- **default** (No Modifier): Restricted to the same package (Package-Private)
- **private**: Strictly accessible only within the same class

SENG 201 – Object Oriented Programming

2024 – 2025 Fall Lectures – Week 12

Ömer Mintemur



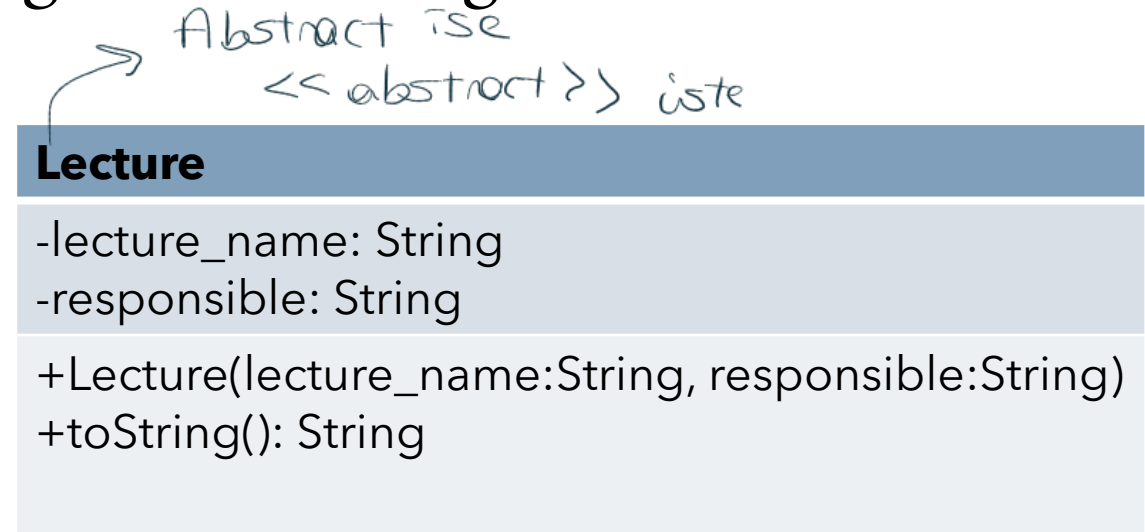
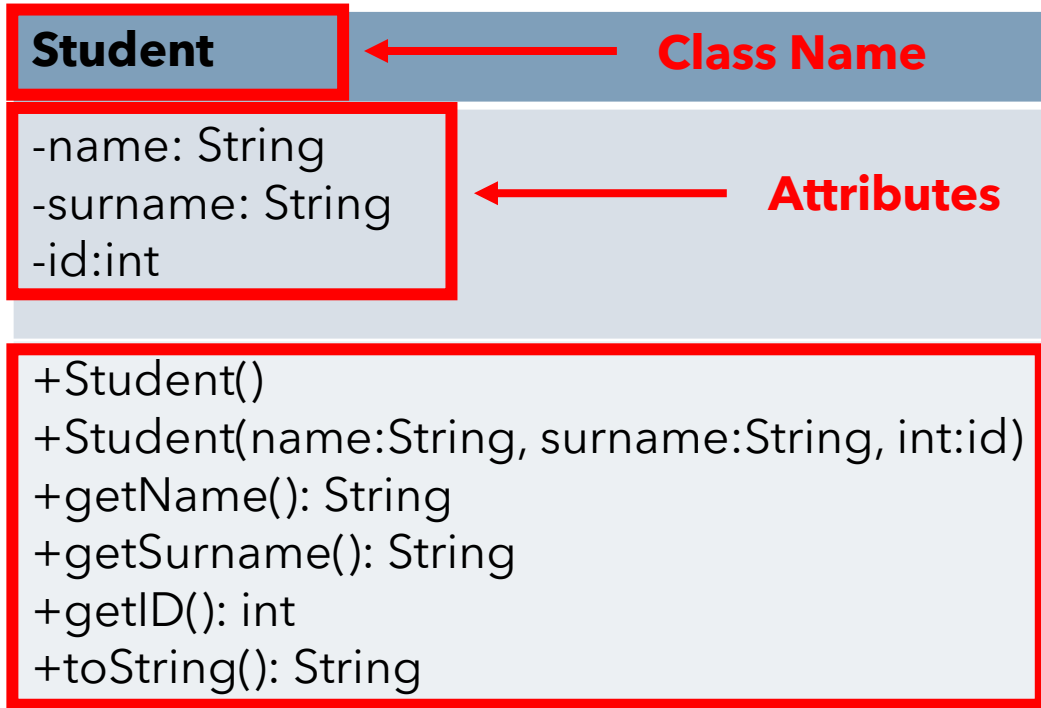
Composition

```
public class Student {  
    // Attributes  
    private String name;  
    private String surname;  
    private int id;  
  
    //Default constructor  
    Student() {}  
    Student(String name, String surname, int id)  
    {  
        this.name = name;  
        this.surname = surname;  
        this.id = id;  
    }  
  
    public String getName() {return this.name;}  
    public String getSurname() {return this.surname;}  
    public int getId() {return this.id;}  
    public String toString() {return this.name + " " + this.surname+" "+this.id;}  
}
```

```
public class Lecture {  
    private String lecture_name;  
    private String responsible;  
    Lecture(String lecture_name,String responsible)  
    {  
        this.lecture_name = lecture_name;  
        this.responsible = responsible;  
    }  
  
    public String toString()  
    {return this.lecture_name  
    + " " +this.responsible;}  
}
```

We want to make relationship between Student and Lecture classes so that «Every lecture has students»

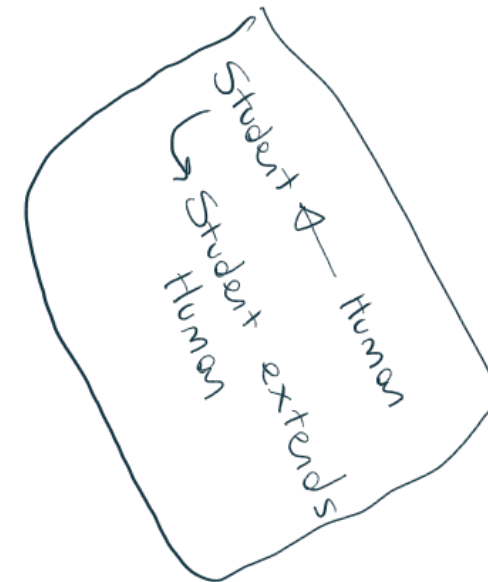
Composition (Unified Modeling Language (UML) Diagrams)



- (negative) sign means private
- + (positive) sign means public
- # (hashtag) sign means protected
- ~ means default

Methods

implements



Composition (Unified Modelling Language (UML) Diagrams)

Student

-name: String
-surname: String
-id: int

+Student()
+Student(name:String, surname:String, int:id)
+getName(): String
+getSurname(): String
+getID(): int
+toString(): String

Lecture

-lecture_name: String
-responsible: String
-students[]: Student

+Lecture(lecture_name:String, responsible:String)
+toString(): String



Every Lecture has students

Composition

```
public class Lecture {  
    private String lecture_name;  
    private String responsible;  
    private Student[] students;  
    Lecture(String lecture_name, String responsible, Student[] st)  
    {  
        this.lecture_name = lecture_name;  
        this.responsible = responsible;  
        this.students = st;  
    }  
    public String toString()  
    {  
        return this.lecture_name  
        + " " + this.responsible;  
    }  
}
```

Lecture has Students

→ override

private Student[] students;



Composition

- Composition is a design principle where an object is made up of other objects
- It models a "has-a" relationship
- **Reusability**: Reuse existing classes by combining them
- **Flexibility**: Changes in one part don't require changes in the composed class
- **Better Design**: Promotes modular and maintainable code

Composition

```
public class Lecture {  
    private String lecture_name;  
    private String responsible;  
    private Student[] students;  
    Lecture(String lecture_name, String responsible, Student[] st)  
    {  
        this.lecture_name = lecture_name;  
        this.responsible = responsible;  
        this.students = st;  
    }  
    public String toString()  
    {return this.lecture_name  
    + " " + this.responsible;}  
  
    public void showStudents()  
    {  
        for(Student s : this.students)  
        {  
            System.out.println(s.toString());  
        }  
    }  
}
```

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    Student[] students = new Student[3];  
    students[0] = new Student("John", "Doe", 1);  
    students[1] = new Student("Jane", "Doe", 2);  
    students[2] = new Student("Mike", "Run", 3);  
  
    Lecture L = new Lecture("SENG201", "Omer", students);  
    System.out.println(L.toString());  
    L.showStudents();  
}
```

```
run:  
SENG201 Omer  
John Doe 1  
Jane Doe 2  
Mike Run 3  
BUILD SUCCESSFUL (total time: 0 seconds)
```


Abstract Classes

In Java, a subclass is a class that inherits from another class, known as the superclass (or parent class). Subclasses allow you to reuse and extend the functionality of the superclass, which is a key concept in object-oriented programming (OOP).inheritance extension and override

- **Definition:**

- Abstract classes are classes that cannot be instantiated and are meant to be extended by other classes. They serve as a blueprint for subclasses.

- **Purpose:**

- They are used to define common properties and methods that can be shared among related classes while leaving implementation details to the subclasses
- Abstract classes can have abstract methods (**methods without a body**) that must be implemented by subclasses
- They can also have concrete (**fully implemented**) methods to provide default behavior to subclasses
- Abstract classes **can have constructors**, which are called during the instantiation of subclasses

```

public class Main {
    public static void main(String[] args) {
        Subclass obj = new Subclass();
        obj.name = "Alice"; // Inherited from Superclass
        obj.age = 25;      // Defined in Subclass

        obj.displayName(); // Inherited method
        obj.displayAge();  // Subclass-specific method
    }
}

class Superclass {
    String name;

    void displayName() {
        System.out.println("Name: " + name);
    }
}

class Subclass extends Superclass {
    int age;

    void displayAge() {
        System.out.println("Age: " + age);
    }
}

```

Aggregation
 Department → Teacher

Abstract Classes

- **When to use:**

- When some methods in a class must be implemented by subclasses while others can have default implementations
- To enforce a contract for subclasses to implement specific behavior

```
public class Animal {  
    private String name;  
    private int age;  
  
    public void noise()  
    {  
        System.out.println("Animal");  
    }  
}
```

```
public class Cat extends Animal{  
  
    //Default Constructor  
    Cat() {}  
    @Override  
    public void noise()  
    {  
        System.out.println("I am a Cat");  
    }  
}
```

```
public static void main(String[] args) {  
    Cat c = new Cat();  
    c.noise();  
}
```

Abstract Classes

```
public static void main(String[] args) {  
    Cat c = new Cat();  
    c.noise();  
    Animal a = new Animal();  
    a.noise();  
}
```

No need to instantiated
Animal class



Make your class abstract

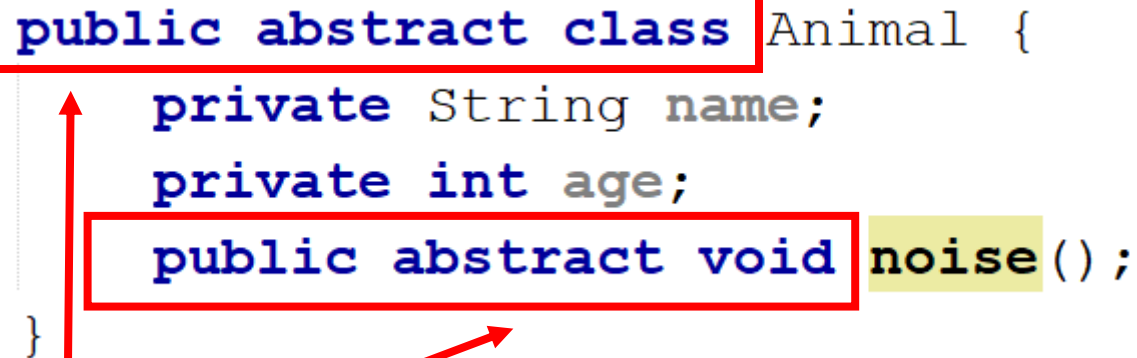


```
public abstract class Animal {  
    private String name;  
    private int age;  
    public void noise()  
    {  
        System.out.println("Animal");  
    }  
}
```

```
public static void main(String[] args) {  
    Cat c = new Cat();  
    c.noise();  
    Animal a = new Animal();  
    a.noise();  
}
```

Abstract Classes

```
public abstract class Animal {  
    private String name;  
    private int age;  
    public abstract void noise();  
}
```



Make your class and methods abstract

```
public class Cat extends Animal {  
    //Default Constructor  
    Cat() {}  
}
```

Now, you must implement the *noise* method

```
public class Cat extends Animal {  
    //Default Constructor  
    Cat() {}  
    @Override  
    public void noise() {  
        System.out.println("I am a Cat");  
    }  
}
```

Abstract Classes

Aspect	Abstract Class	Abstract Method
<i>Definition</i>	A class marked with the abstract keyword.	A method marked with the abstract keyword
<i>Purpose</i>	Serves as a blueprint for other classes	Specifies behavior that subclasses must define
<i>Implementation</i>	May have both abstract and concrete methods	Cannot have an implementation (no body)
<i>Usage</i>	Can have constructors and fields to support subclasses	Can only exist in abstract classes or interfaces
<i>Instantiation</i>	Cannot be instantiated directly	Must be implemented in concrete subclasses

main içinde yazılabilir
Animal a = new Animal();

Abstract Classes

```
abstract class Vehicle {  
    private String brand;  
    // Constructor  
    public Vehicle(String brand) {  
        this.brand = brand;  
    }  
    // Abstract method for fuel consumption  
    // must be implemented by subclasses  
    abstract void fuelConsumption();  
    // Abstract method for number of wheels  
    // must be implemented by subclasses  
    abstract int numberOfWheels();  
    // Concrete method - shared by all vehicles  
    public void start() {  
        System.out.println(brand + " is starting.");  
    }  
    // Concrete method - shared by all vehicles  
    public void stop() {  
        System.out.println(brand + " is stopping.");  
    }  
}
```

All vehicles conform to the Vehicle abstract class, ensuring consistency in the system

Concrete methods like start and stop avoid code duplication

Specific behaviors for each type of vehicle are implemented in their respective subclasses

Abstract Classes

```
class Car extends Vehicle {  
    public Car(String brand) {  
        super(brand);  
    }  
    @Override  
    void fuelConsumption() {  
        System.out.println("Car consumes "  
            + "8 liters of fuel per 100 km.");  
    }  
    @Override  
    int numberOfWheels() {  
        return 4;  
    }  
}
```

```
// Subclass for Bicycle  
class Bicycle extends Vehicle {  
    public Bicycle(String brand) {  
        super(brand);  
    }  
    @Override  
    void fuelConsumption() {  
        System.out.println("Bicycle doesn't "  
            + "consume fuel.");  
    }  
    @Override  
    int numberOfWheels() {  
        return 2;  
    }  
}
```

```
// Subclass for Airplane  
class Airplane extends Vehicle {  
    public Airplane(String brand) {  
        super(brand);  
    }  
    @Override  
    void fuelConsumption() {  
        System.out.println("Airplane consumes "  
            + "500 liters of fuel per hour.");  
    }  
    @Override  
    int numberOfWheels() {  
        return 10;  
    }  
}
```

Abstract → extend //

Abstract Classes

```
public static void main(String[] args) {  
    Vehicle car = new Car("Toyota");  
    Vehicle bicycle = new Bicycle("Bianchi");  
    Vehicle airplane = new Airplane("Boeing");  
    car.start();  
    car.fuelConsumption();  
    System.out.println("Car has "  
        + car.numberOfWheels()  
        + " wheels.\n");  
    bicycle.start();  
    bicycle.fuelConsumption();  
    System.out.println("Bicycle has "  
        + bicycle.numberOfWheels()  
        + " wheels.\n");  
    airplane.start();  
    airplane.fuelConsumption();  
    System.out.println("Airplane has "  
        + airplane.numberOfWheels()  
        + " wheels.\n");  
}
```

Abstract Classes – Key Points in the Example

- **Abstract Class - Vehicle**

- Acts as a blueprint for all types of vehicles
- Enforces common structure through abstract methods (fuelConsumption and numberOfWheels)
- Provides shared functionality (start and stop) as concrete methods

- **Subclasses**

- Car, Bicycle, and Airplane extend Vehicle and implement the abstract methods to define their unique behaviors

- **Advantages**

- **Code Reusability:** Shared behaviors (like start and stop) are implemented once in the Vehicle class
- **Extensibility:** Adding a new vehicle type (e.g., Boat) only requires creating a new subclass

Interfaces

- **Definition:**

- An interface is a blueprint for a class that defines a contract (methods and constants) that implementing classes must follow
- Use the `interface` keyword

- **Methods**

- All methods in an interface are public and abstract by default

- **Implementation**

- A class can implement multiple interfaces (supports multiple inheritance)

Interfaces

İçine tanımlanan değerele otomatik olarak public static final olur -

How to define an interface

```
// Define an interface
```

```
interface Animal {
```

```
    void sound(); // Abstract method
```

```
}
```

```
// Implement the interface in a class
```

```
class Dog implements Animal {
```

```
    // Provide implementation for the sound() method
```

```
    public void sound() {
```

```
        System.out.println("The dog says: Woof Woof!");
```

```
    }
```

```
}
```

Class implements the interface

```
public static void main(String[] args) {
```

```
    Animal myDog = new Dog();
```

```
    myDog.sound(); // Call the implemented method
```

```
}
```

interface → implements

İçerdiği metotlar otomatik olarak public ve abstract olur

Interfaces (Scenario: Flying and Swimming Animals)

- **Problem**

- Some animals can fly, some can swim, and some can do both

```
// Define the Flyable interface
interface Flyable {
    void fly();
}
```

```
// Define the Swimmable interface
interface Swimmable {
    void swim();
}
```

```
// Define an abstract class for common animal characteristics
abstract class Animal {
    String name;

    Animal(String name) {
        this.name = name;
    }

    abstract void sound(); // Abstract method
}
```

abstract içinde tanımlanmış interface içinde tanımlanaktan daha avantajlı değer değiştirmek istiyorsanız

Interfaces (Scenario: Flying and Swimming Animals)

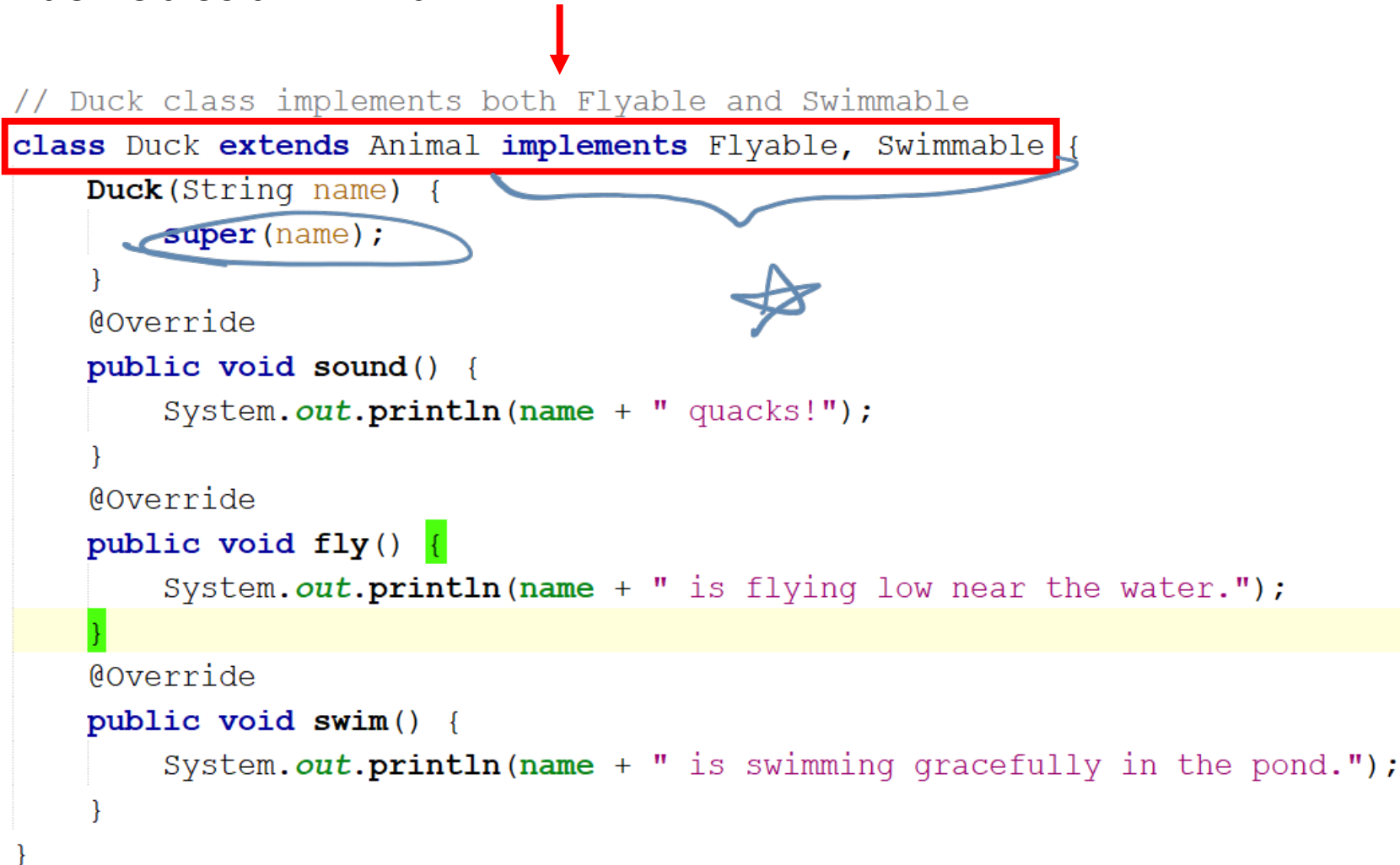
Both class extend Animal and implements a specific interface

```
// Bird class implements Flyable
class Bird extends Animal implements Flyable {
    Bird(String name) {
        super(name);
    }
    @Override
    public void sound() {
        System.out.println(name + " chirps!");
    }
    @Override
    public void fly() {
        System.out.println(name + " is flying high in the sky.");
    }
}
```

```
// Fish class implements Swimmable
class Fish extends Animal implements Swimmable {
    Fish(String name) {
        super(name);
    }
    @Override
    public void sound() {
        System.out.println(name + " makes bubbling sounds.");
    }
    @Override
    public void swim() {
        System.out.println(name + " is swimming in the water.");
    }
}
```

Interfaces (Scenario: Flying and Swimming Animals)

**Since a Duck can both Fly and Swim. It implements both interfaces.
A Duck is also an Animal**



```
// Duck class implements both Flyable and Swimmable
class Duck extends Animal implements Flyable, Swimmable {
    Duck(String name) {
        super(name);
    }
    @Override
    public void sound() {
        System.out.println(name + " quacks!");
    }
    @Override
    public void fly() {
        System.out.println(name + " is flying low near the water.");
    }
    @Override
    public void swim() {
        System.out.println(name + " is swimming gracefully in the pond.");
    }
}
```

Interfaces (Scenario: Flying and Swimming Animals)

```
// Create instances of each animal
Bird bird = new Bird("Parrot");
Fish fish = new Fish("Goldfish");
Duck duck = new Duck("Duck");

// Demonstrate behaviors
bird.sound();
bird.fly();

fish.sound();
fish.swim();

duck.sound();
duck.fly();
duck.swim();
```

```
run:
Parrot chirps!
Parrot is flying high in the sky.
Goldfish makes bubbling sounds.
Goldfish is swimming in the water.
Duck quacks!
Duck is flying low near the water.
Duck is swimming gracefully in the pond.
```


Interfaces vs Abstract Classes

Aspect	Abstract Classes	Interfaces
Purpose	To provide a base class for related classes	To define a contract for unrelated classes
Inheritance	Supports single inheritance	Supports multiple inheritance
Attributes	Can have instance variables	Fields are always public static final (constants)
Constructors	Can have constructors	Cannot have constructors
Access Modifiers	Methods can have various access levels	Methods are always public
When to Use	Use when classes share a common structure and behavior	Use when you need to define behavior without shared structure

Tablodaki maddeleri şu şekilde açıklayabilirim:

1. Purpose (Amaç)

- **Abstract Classes (Soyut Sınıflar):**

Soyut sınıflar, ilişkili sınıflar için bir temel sınıf oluşturur. Ortak özelliklere ve davranışlara sahip sınıflar arasında kod tekrarını önlemek için kullanılır.

- **Interfaces (Arayüzler):**

Arayüzler, birbirinden bağımsız (ilişkisiz) sınıflar için bir sözleşme tanımlar. Sınıfların belirli bir davranışı uygulamalarını zorunlu kılmak için kullanılır.

2. Inheritance (Kalıtım)

- **Abstract Classes:**

Soyut sınıflar sadece *tekli kalıtım* destekler. Bir sınıf sadece bir soyut sınıfı miras alabilir.

- **Interfaces:**

Arayüzler *çoklu kalıtım* destekler. Bir sınıf birden fazla arayüzü implemente edebilir.

3. Attributes (Öznitelikler)

- **Abstract Classes:**

Soyut sınıflar *örnek değişkenleri* (*instance variables*) içerebilir. Bu değişkenler, farklı erişim belirleyicilere (*private*, *protected*, *public*) sahip olabilir.

- **Interfaces:**

Arayüzlerde tanımlanan tüm alanlar *public static final* olarak kabul edilir. Yani, bunlar sabit değerlerdir (*constants*) ve değiştirilemez.

4. Constructors (Yapıcı Metotlar)

- **Abstract Classes:**

Soyut sınıflar yapıcı metotlar (*constructors*) içerebilir. Bu yapıcılar, alt sınıflar oluşturulurken soyut sınıfın bazı başlangıç değerlerini ayarlamak için kullanılabilir.

- **Interfaces:**

Arayüzler yapıcı metot içeremez. Çünkü arayüzler örneklenemez ve bir nesne oluşturulamaz.

5. Access Modifiers (Erişim Belirleyiciler)

- **Abstract Classes:**

Soyut sınıflarda metotlar çeşitli erişim belirleyicilere (*public*, *protected*, *private*) sahip olabilir.

- **Interfaces:**

Arayüzlerdeki metotlar varsayılan olarak *public* ve *abstract* kabul edilir. Farklı erişim belirleyiciler kullanılamaz.

6. When to Use (Kullanım Zamanı)

- **Abstract Classes:**

Eğer sınıflar arasında ortak bir yapı ve davranış gerekiyorsa (örneğin, hem özellikler hem de bazı varsayılan metotlar paylaşılacaksa), soyut sınıflar kullanılır.

- **Interfaces:**

Eğer sınıfların belirli bir davranışı uygulamasını zorunlu kılmak istiyorsanız (ortak bir yapı gerekmiyorsa), arayüzler tercih edilir.

Soruların varsa devam edebilirim!

SENG 201 – Object Oriented Programming

2024 – 2025 Fall Lectures – Week 13

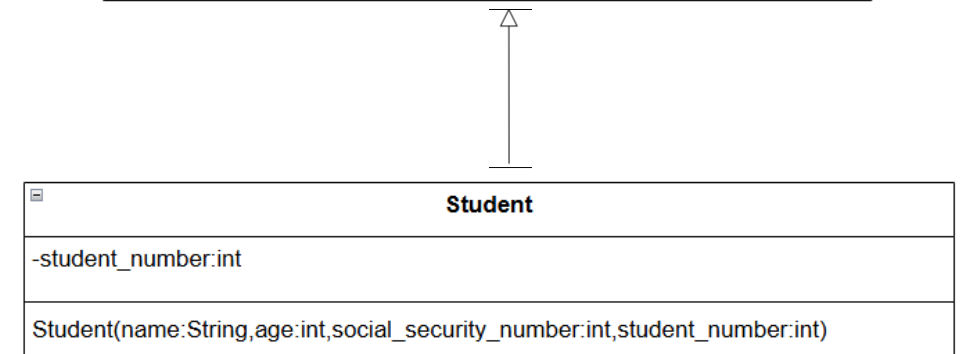
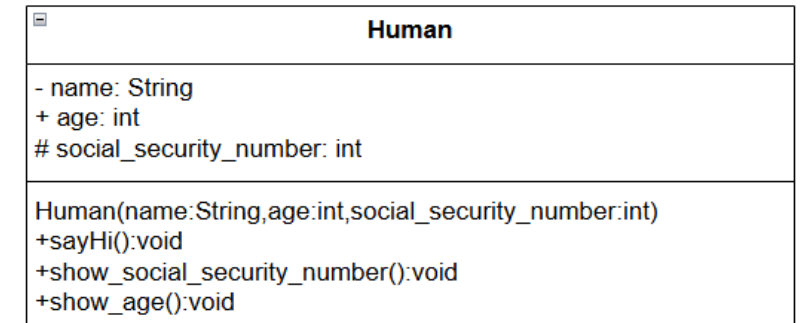
Ömer Mintemur



UML Diagrams

```
public class Human {  
    private String name;  
    public int age;  
    protected int social_security_number;  
  
    Human(String name,int age, int social_security_number)  
    {  
        this.name = name;  
        this.age = age;  
        this.social_security_number = social_security_number;  
    }  
  
    public void sayHi()  
    {  
        System.out.println("Hi, my name is " + this.name);  
    }  
  
    public void show_social_security_number()  
    {  
        System.out.println("Social Security Number "  
            + this.social_security_number);  
    }  
  
    public void show_age()  
    {  
        System.out.println("My age is " + this.age);  
    }  
}
```

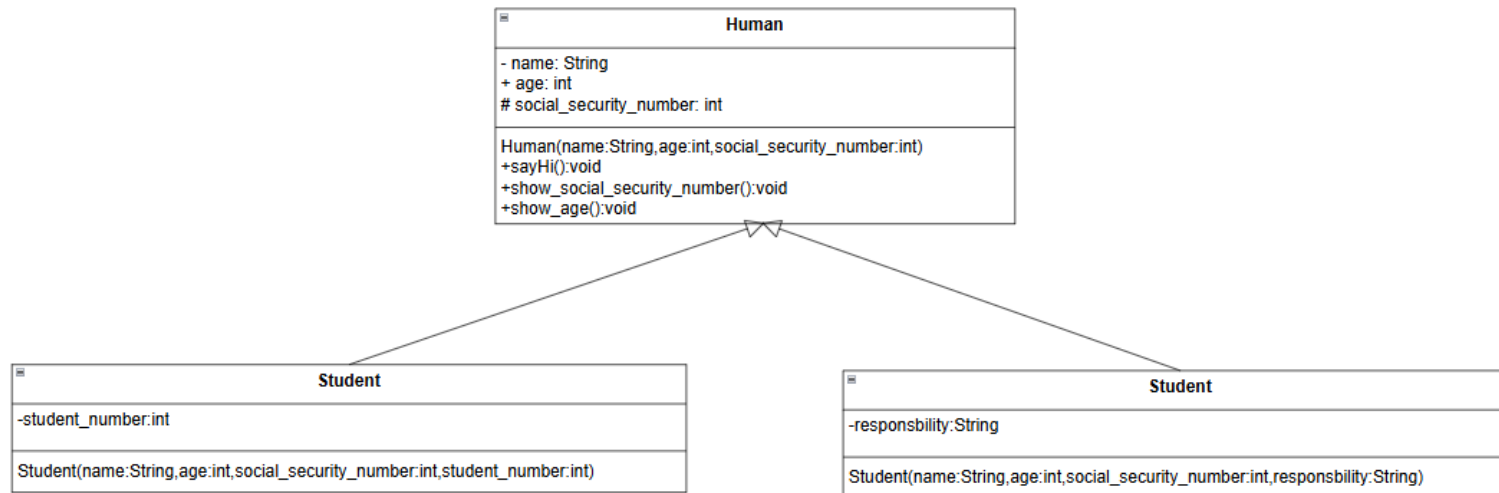
```
public class Student extends Human {  
  
    private int student_number;  
  
    Student(String name, int age,  
            int social_security_number,  
            int student_number)  
    {  
        super(name,age,social_security_number);  
        this.student_number = student_number;  
    }  
}
```



UML Diagrams

```
public class Student extends Human {  
    private int student_number;  
  
    Student(String name, int age,  
            int social_security_number,  
            int student_number)  
    {  
        super(name, age, social_security_number);  
        this.student_number = student_number;  
    }  
}
```

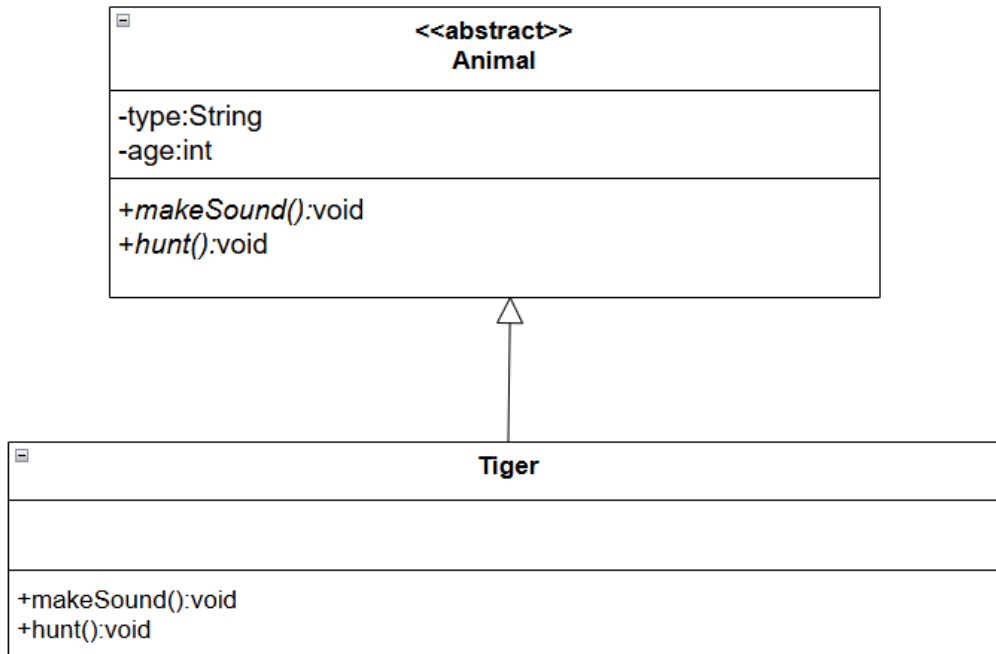
```
public class Sophomore_Student extends Human {  
  
    private String responsibility;  
  
    Sophomore_Student(String name, int age,  
                      int social_security_number,  
                      String responsibility)  
    {  
        super(name, age, social_security_number);  
        this.responsibility = responsibility;  
    }  
}
```



UML Diagrams

```
public abstract class Animal {  
    private String type;  
    private int age;  
    public abstract void makeSound();  
    public abstract void hunt();  
}
```

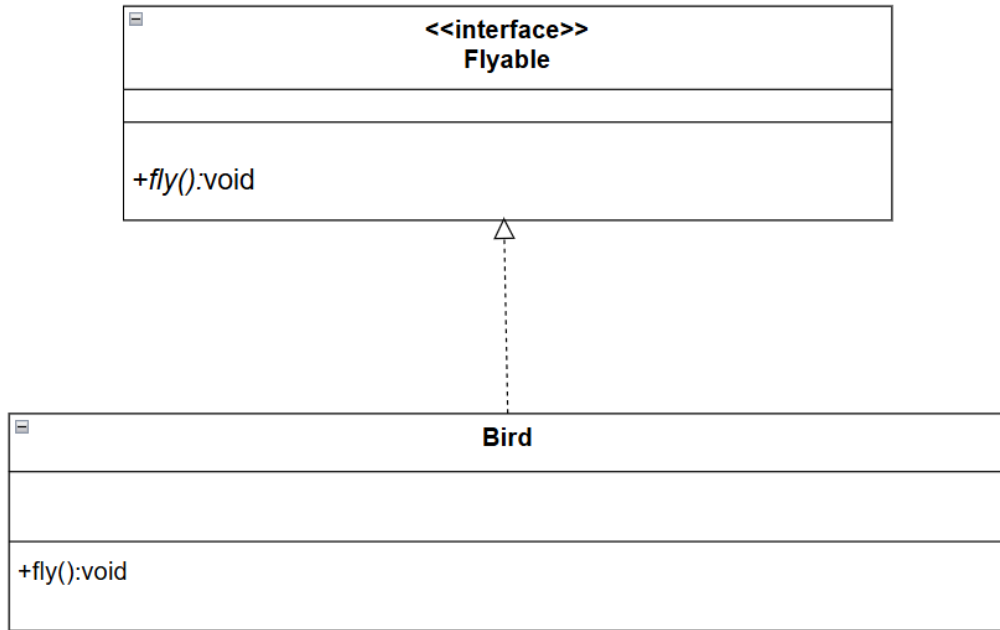
```
public class Tiger extends Animal {  
    @Override  
    public void makeSound() {  
        System.out.println("Tiger sound is: Wooor");  
    }  
    @Override  
    public void hunt() {  
        System.out.println("I hunt by running");  
    }  
}
```



UML Diagrams

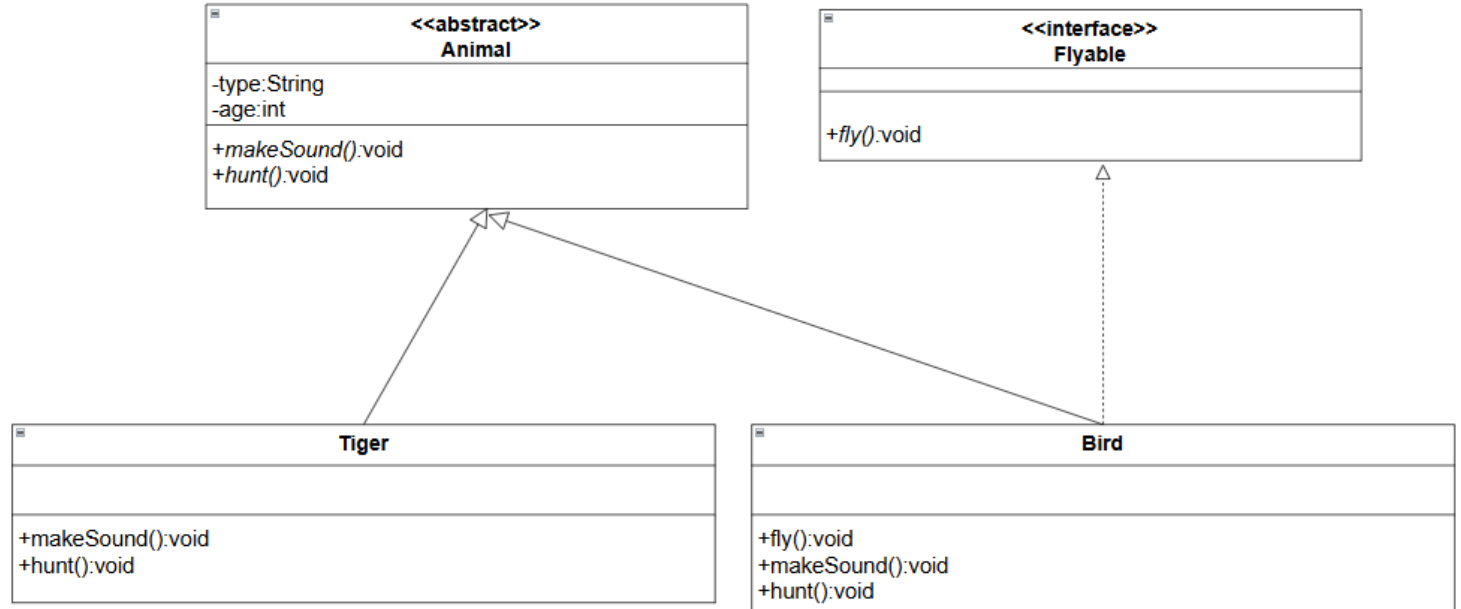
```
public interface Flyable {  
    void fly();  
}
```

```
public class Bird implements Flyable {  
    public void fly()  
    {  
        System.out.println("I am flying");  
    }  
}
```



UML Diagrams

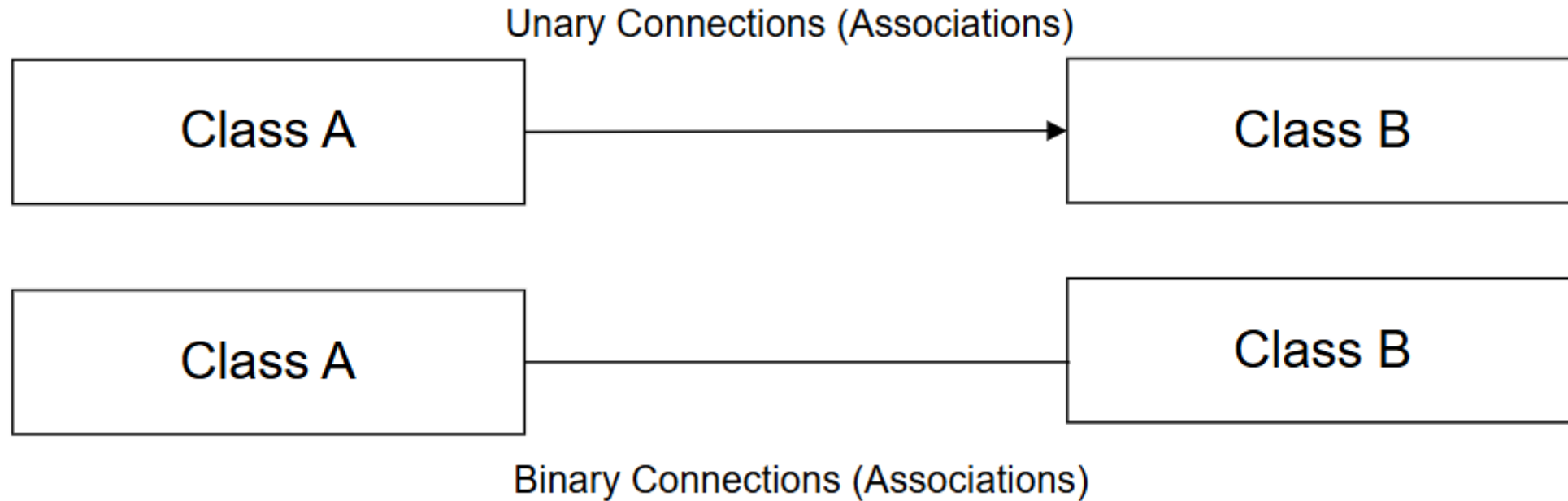
```
public class Bird extends Animal implements Flyable {  
    public void fly()  
    {  
        System.out.println("I am flying");  
    }  
    @Override  
    public void makeSound() {  
        System.out.println("Bird sound: ");  
    }  
    @Override  
    public void hunt() {  
        System.out.println("I eat spiders");  
    }  
}
```



UML Diagrams (Arrow Types)

- Inheritance (Generalization)
 - **Arrow Type:** A **solid line** with a **hollow triangle** pointing to the parent class
 - **Purpose:** Represents a **subclass inheriting from a superclass**
- Abstract Class
 - **Arrow Type:** Same as inheritance, i.e., a **solid line** with a **hollow triangle** pointing to the abstract class
 - **Purpose:** Indicates that the subclass inherits from an abstract class
 - **Key Note:** Abstract classes cannot be instantiated, only extended
- Interface Implementation
 - **Arrow Type:** A **dashed line** with a **hollow triangle** pointing to the interface
 - **Purpose:** Indicates that a class implements an interface
 - **Key Note:** A class can implement multiple interfaces

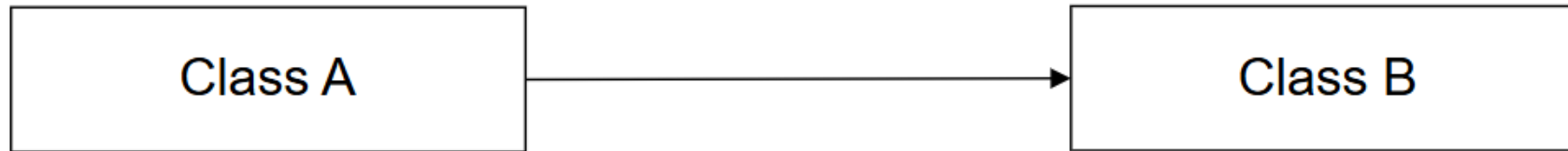
UML Diagrams (Connections Between Classes)



UML Diagrams (Connections Between Classes)

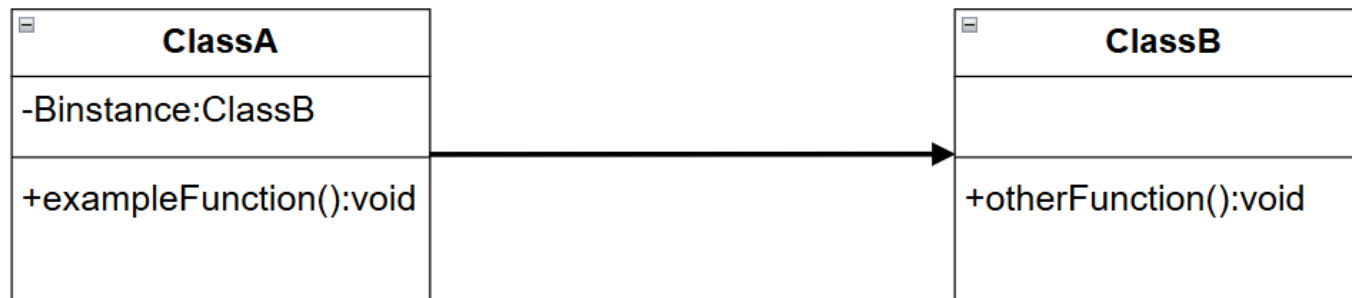
- **Unary Connection (Associations)**

Unary Connections (Associations)



ClassA knows about ClassB

ClassB does not know about ClassA



UML Diagrams (Connections Between Classes)

- There can be 3 types of associations:

- **Association**

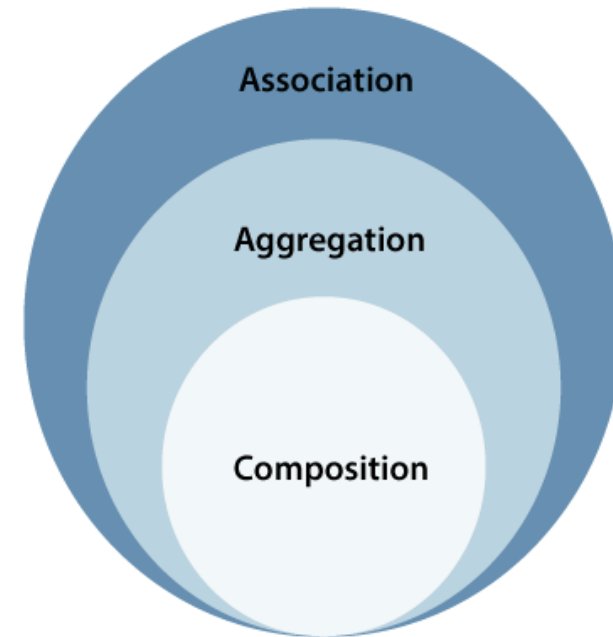
- Most general

- **Aggregation**

- Whole part relationships

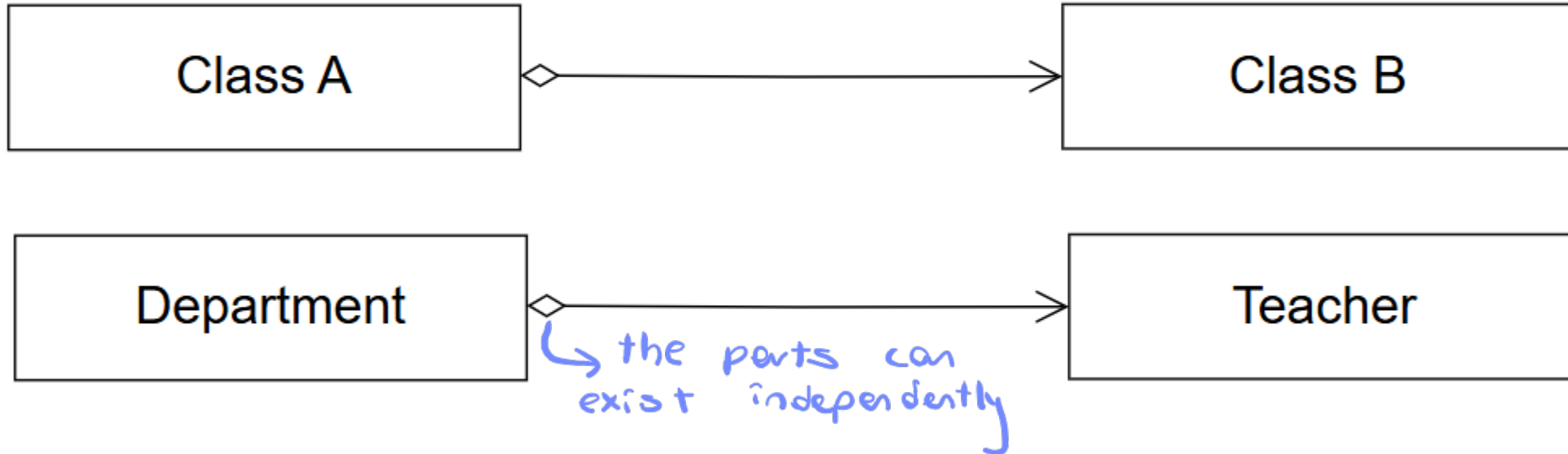
- **Composition**

- Whole part relationships



UML Diagrams (Connections Between Classes)

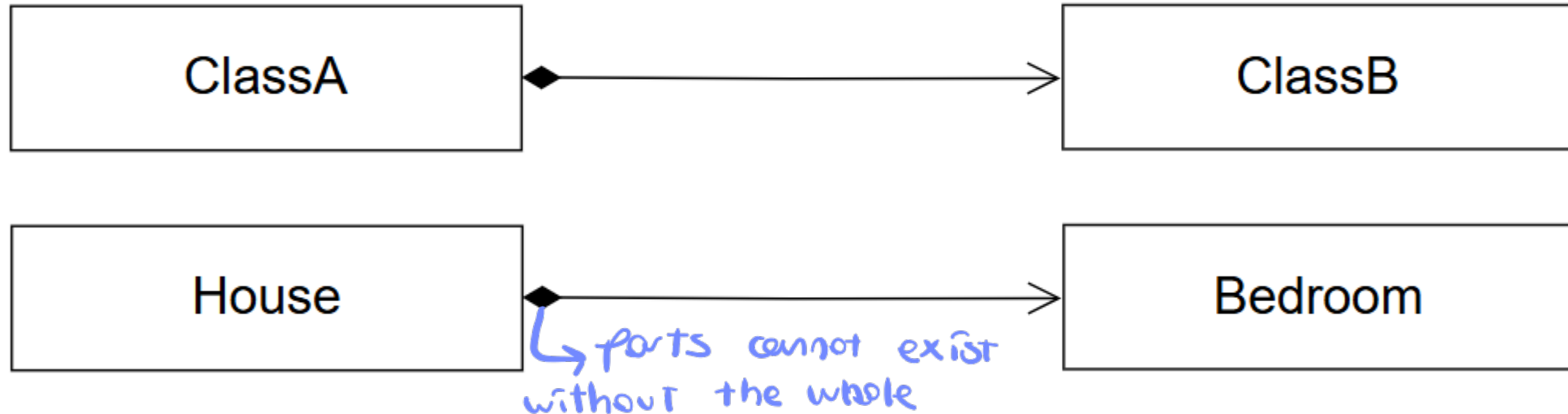
- **Aggregation**



Definition: A stronger form of association. One class (whole) can contain objects of another class (parts), but the parts can exist independently of the whole
Teachers can exist independently outside of a department

UML Diagrams (Connections Between Classes)

- **Composition**



Definition: Definition: A stronger relationship where one class (whole) owns the objects of another class (parts). The lifecycle of the part depends on the whole. Represents a "part-of" relationship; parts cannot exist without the whole
If a house is destroyed, its rooms are destroyed too

UML Diagrams (Connections Between Classes)

Feature	Association	Aggregation	Composition
Relationship	Weak «Uses»	Strong «has-a»	Strong «part-of»
Lifecycle	Independent	Independent	Dependent
Example	Student - Course	Department - Teacher	House - Rooms

UML Diagrams (Connections Between Classes)

- **Binary Connection (Associations)**



Two Classes Involved: Binary association connects exactly two classes.

Multiplicity: Specifies how many instances of one class are related to how many instances of another class

→ **0..1** an optional instance (zero or one)

→ **n** exactly n instances

→ **0..*** 0 or many instances

→ **1..*** one or many instances

→ **n..m** n to m instances

UML Diagrams (Connections Between Classes)

- **0..1 (Optional Instance) in Binary Association**

- This type of relationship is commonly used when an object is *not required* to have a corresponding instance in the other class. It is a way to model situations where the relationship between the two classes is not mandatory



Consider the relationship between a Driver and a License:

- A Driver may or may not have a License.
- If a Driver does have a License, it's exactly one License.

Here, the association between Driver and License is 0..1, meaning:

- A driver can either have a license (1), or they may not have one (0).

UML Diagrams (Connections Between Classes)

- **Exactly n instances in Binary Association**
- An **exactly n instances** relationship means that a specific number of instances from one class must be associated with exactly one instance of another class



- Enforcing exactly n instances ensures that associations conform to domain rules (e.g., a car cannot function with fewer or more than 4 wheels).
- Code logic must validate and restrict the addition or removal of associated objects to match the n constraint

UML Diagrams (Connections Between Classes)

- **0..* (zero or many) instances**

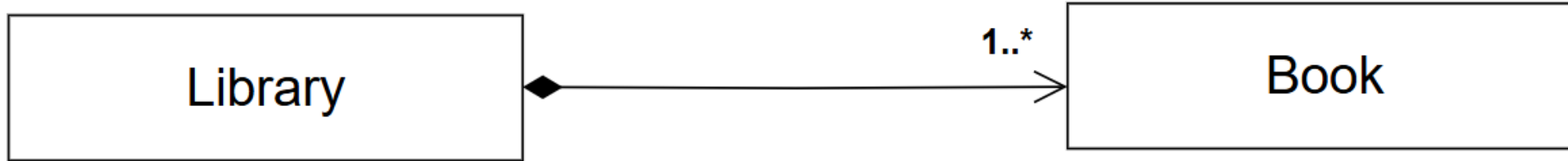


- **Library** can have **zero** or **more books**.
 - If the library is newly built, it may have no books (0)
 - As books are acquired, the number grows (1, 2, ..., n)
- The «whole» can exist without any associated «parts»
- There's no upper limit to the number of associated instances
- The parts may exist independently (books can exist outside the library)

Shopping Cart and Items
Teacher and Students in a course
Folder and Files

UML Diagrams (Connections Between Classes)

- **1..* (one or many) instances**



Composition (strong ownership):

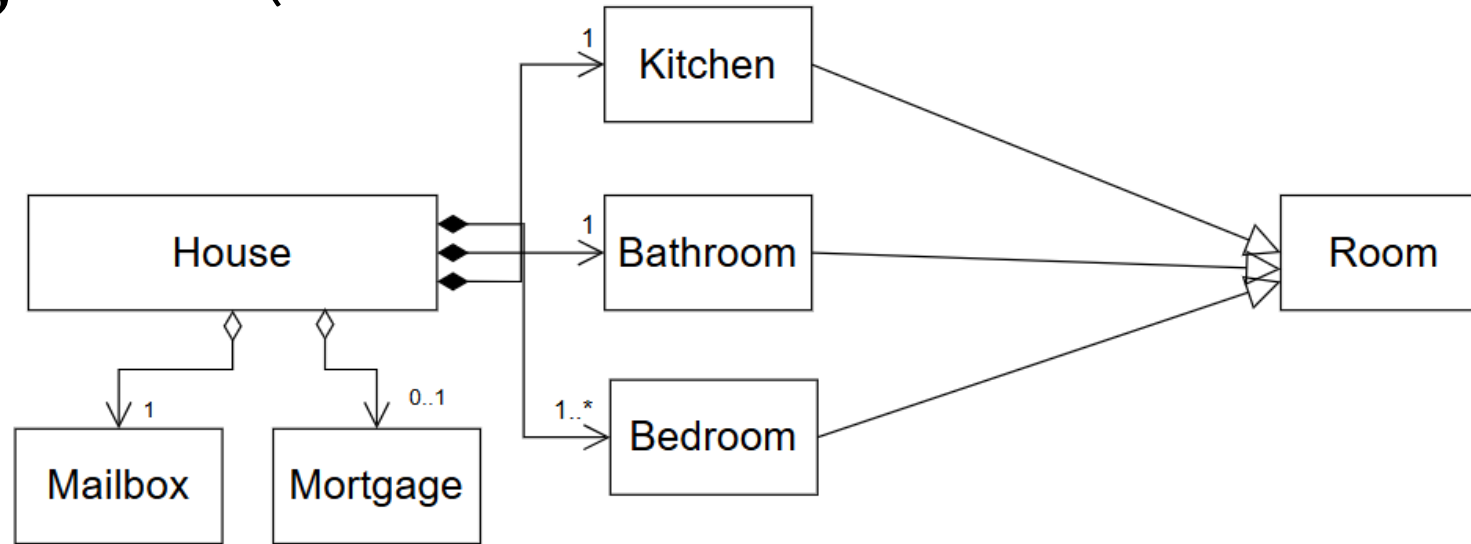
- The **library** owns and controls the **book**'s lifecycle. If the **library** is deleted, the **book** will also cease to exist
- The **book** is an integral part of the **library**, and cannot exist outside of it in a meaningful way



Aggregation (weaker relationship)

- The **library** contains **books**, but **books** can exist independently of the **library**.
- **Books** can exist without the **library**, but still, the **library** aggregates them

UML Diagrams (Connections Between Classes)



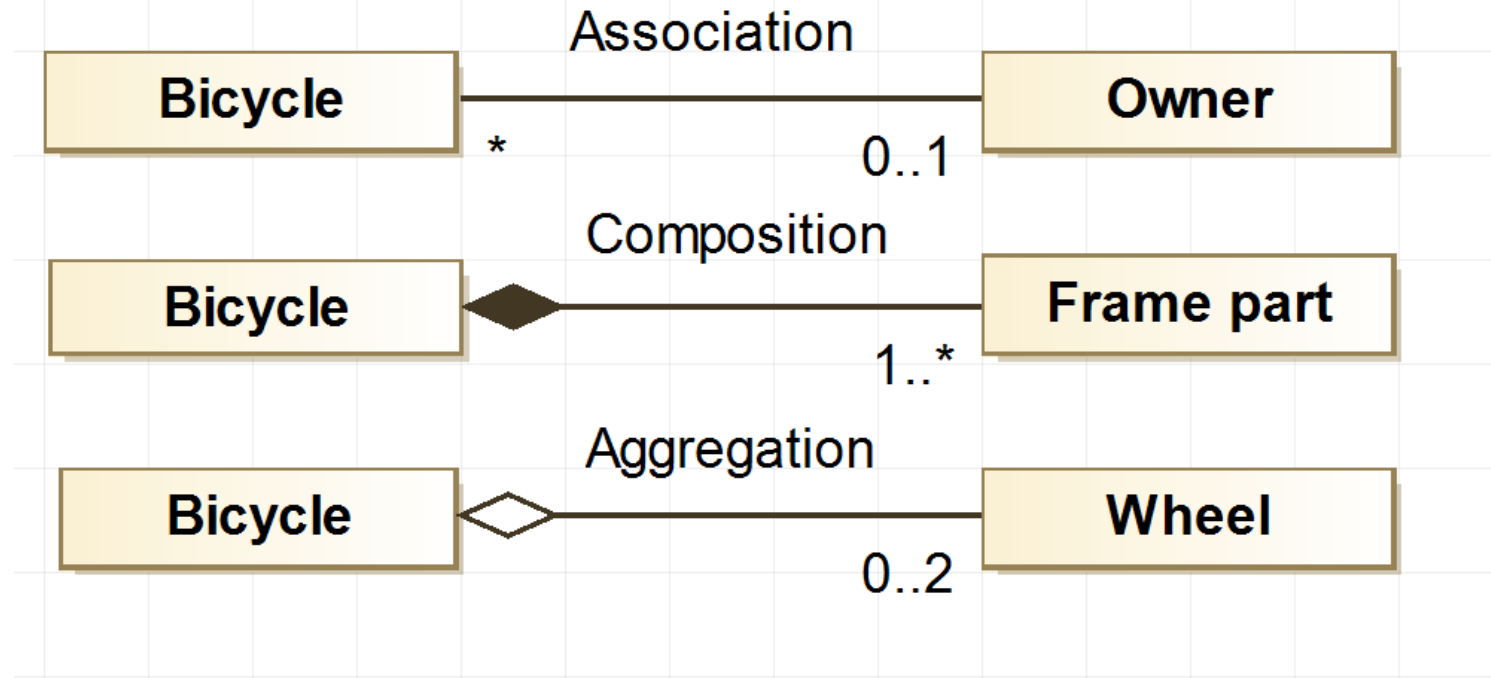
Kitchen - Bathroom - Bedroom extend Room

House can have 1 Kitchen - 1 Bathroom - 1 or more Bedrooms.
Kitchen, Bathroom and Bedroom can not exist independently.

House can have one Mailbox. But Mailbox can exist independently.

House can have zero or 1 Mortgage. But Mortgage can exist independently.

UML Diagrams (Connections Between Classes)



An Owner can have multiple Bicycle

A Bicycle can be owned by zero or 1 Owner

A Bicycle should have at least one or more Frame Part. Frame Part can not exist independently

A Bicycle can have zero or 2 Wheels. Wheel can exist independently

ANKARA YILDIRIM BEYAZIT UNIVERSITY

SENG 201 OBJECT ORIENTED PROGRAMMING

LAB 2

02.10.2024

QUESTIONS

- 1) Write a program that prints all the prime numbers between 2 and 1500 (50).

It is shown until 20

```
run:
2
3
4
5
7
11
13
17
19
BUILD SUCCESSFUL (total time: 0 seconds)
```

- 2) Write a program that reads integers, finds the smallest of them, and count its occurrences
Assume that 0 (zero) is the terminal value.

```
Enter a number 5
Enter a number 2
Enter a number 2
Enter a number 1
Enter a number 2
Enter a number 3
Enter a number 2
Enter a number 0
Min Number is 1 Count is 1
```

SENG 201 – Object Oriented Programming

2024 – 2025 Fall Lectures - Week 14

Ömer Mintemur



Generics

```
public class Cat {  
    private String name;  
    private int age;  
    Cat(String name, int age)  
    {  
        this.name = name;  
        this.age = age;  
    }  
    public String getName()  
    {  
        return this.name;  
    }  
    public int getAge()  
    {  
        return this.age;  
    }  
    public String toString()  
    {  
        return "Name is " + this.getName()  
            + "\nAge is " + this.getAge();  
    }  
}
```

```
public class Dog {  
    private String name;  
    private int age;  
    Dog(String name, int age)  
    {  
        this.name = name;  
        this.age = age;  
    }  
    public String getName()  
    {  
        return this.name;  
    }  
    public int getAge()  
    {  
        return this.age;  
    }  
    public String toString()  
    {  
        return "Name is " + this.getName()  
            + "\nAge is " + this.getAge();  
    }  
}
```

Generics

```
public class Show {  
    public void show(Cat c)  
    {  
        System.out.println(c.toString());  
    }  
    public void show(Dog d)  
    {  
        System.out.println(d.toString());  
    }  
}  
  
public static void main(String[] args) {  
    Cat c = new Cat ("Mike", 2);  
    Dog d = new Dog ("John", 1);  
    Show s = new Show();  
    s.show(c);  
    s.show(d);  
}
```

Problem:

What if you have 20 different classes?

First Solution:

You can write 20 different functions for each object.

Second Solution:

You can accept general Object type.
Then you have to do downcasting
(Next Slide)

Generics

```
public class Show {  
    public void show(Object o)  
    {  
        if (o instanceof Cat)  
        {  
            System.out.println(((Cat)o).toString());  
        }  
        if (o instanceof Dog)  
        {  
            System.out.println(((Dog)o).toString());  
        }  
    }  
}
```

```
public static void main(String[] args) {  
    Cat c = new Cat ("Mike", 2);  
    Dog d = new Dog ("John", 1);  
    Show s = new Show();  
    s.show(c);  
    s.show(d);  
}
```

Again, you have to control the type of the Object. You to write 20 different if conditions

Generics

We can use Generics !!!

Type Parameter

```
public class Show <T> {  
  
    public void show(T o)  
    {  
        System.out.println(o.toString());  
    }  
}
```

Explicitly give the type

```
public static void main(String[] args) {  
    Cat c = new Cat ("Mike", 2);  
    Dog d = new Dog ("John", 1);  
    Show<Cat> s_cat = new Show<Cat>();  
    s_cat.show(c);  
    Show<Dog> s_dog = new Show<Dog>();  
    s_dog.show(d);  
}
```

Generics

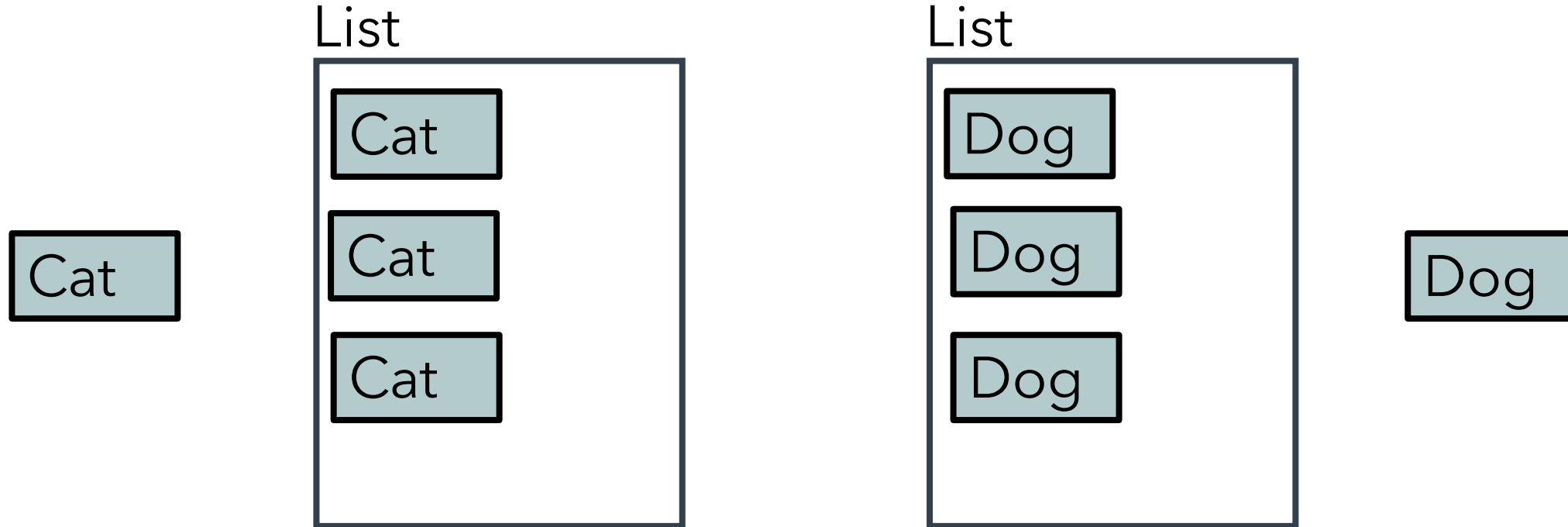
- Generics allow a class or method to operate on objects of various types while ensuring type safety at compile time
- **Type Safety**
 - Prevents accidental operations on incompatible types
- **Code Reusability**
 - Write a single generic class or method to work with multiple data types
- **Avoidance of Type Casting**
 - Eliminates the need for explicit type casting

Generics

```
public class Box<T> {  
  
    private T item;  
  
    public void set (T item)  
    {  
        this.item = item;  
    }  
    public T get()  
    {  
        return this.item;  
    }  
}
```

```
public static void main(String[] args) {  
    Cat c = new Cat ("Mike", 2);  
    Dog d = new Dog ("John", 1);  
    Box<Cat> box_cat = new Box<Cat>();  
    box_cat.set(c);  
    Cat n = box_cat.get();  
    System.out.println(n.toString());  
}
```

Generics



Make a Generic List Class that can hold instances of Dog or Cat classes

Generics

```
public class Cat {
    private String name;
    private int age;
    Cat(String name, int age)
    {
        this.name = name;
        this.age = age;
    }
    public String getName()
    {
        return this.name;
    }
    public int getAge()
    {
        return this.age;
    }
    public String toString()
    {
        return "Name is " + this.getName()
            + "\nAge is " + this.getAge();
    }
}
```

```
public class Dog {
    private String name;
    private int age;
    Dog(String name, int age)
    {
        this.name = name;
        this.age = age;
    }
    public String getName()
    {
        return this.name;
    }
    public int getAge()
    {
        return this.age;
    }
    public String toString()
    {
        return "Name is " + this.getName()
            + "\nAge is " + this.getAge();
    }
}
```


Generics

```
public class MyList <T> {
    private T[] instances;
    private int index = 0;
    MyList()
    {
        // this.instances = new T[10];
        // You must create new array in Object type
        // Then cast it to class <T>
        // This is IMPORTANT !!!
        this.instances = (T[]) new Object[100];
    }
    public void addItem(T item)
    {
        if (this.index == 100)
        {
            System.out.println("Can not add more!!!");
        }
        else{this.instances[this.index++] = item;}
    }
    public void showItem()
    {
        for (T t:this.instances)
        {
            if (t == null){break;}
            System.out.println(t.toString());
        }
    }
}
```

This is not allowed because the exact type of T is unknown at runtime, and Java cannot allocate memory for an unknown type

```
public static void main(String[] args) {
    Cat cat_1 = new Cat("Mike",1);
    Cat cat_2 = new Cat("Sara", 1);
    Cat cat_3 = new Cat("John", 3);
    MyList<Cat> cat_list = new MyList<Cat>();
    cat_list.addItem(cat_1);
    cat_list.addItem(cat_2);
    cat_list.addItem(cat_3);
    cat_list.showItem();
    MyList<Dog> dog_list = new MyList<Dog>();
    Dog dog_1 = new Dog("Jen",1);
    Dog dog_2 = new Dog("Ken", 1);
    Dog dog_3 = new Dog("Ren", 3);
    dog_list.addItem(dog_1);
    dog_list.addItem(dog_2);
    dog_list.addItem(dog_3);
    dog_list.showItem();
}
```

Generics – Upper Bounded

```
public class Animal {  
  
    private String name;  
    private int age;  
  
    Animal(String name, int age)  
    {  
        this.name = name;  
        this.age = age;  
    }  
    public String toString()  
    {  
        return "Name is " + this.name  
            + "\nAge is " + this.age;  
    }  
}
```

```
public class Dog extends Animal {  
    private String name;  
    private int age;  
    Dog(String name, int age)  
    {  
        super(name, age);  
    }  
    public String getName()  
    {  
        return this.name;  
    }  
    public int getAge()  
    {  
        return this.age;  
    }  
}
```

```
public class Cat extends Animal {  
    private String name;  
    private int age;  
    Cat(String name, int age)  
    {  
        super(name, age);  
    }  
    public String getName()  
    {  
        return this.name;  
    }  
    public int getAge()  
    {  
        return this.age;  
    }  
}
```

Generics – Upper Bounded

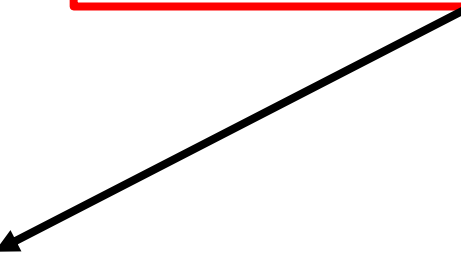
```
public class Tiger {  
    private String name;  
    private int age;  
    Tiger(String name, int age)  
    {  
        this.name = name;  
        this.age = age;  
    }  
    public String getName()  
    {  
        return this.name;  
    }  
    public int getAge()  
    {  
        return this.age;  
    }  
    public String toString()  
    {  
        return "Name is " + this.getName()  
            + "\nAge is " + this.getAge();  
    }  
}
```

```
public class UpperBounded <T extends Animal> {  
  
    public void show(T item)  
    {  
        System.out.println(item.toString());  
    }  
}
```

Generics – Upper Bounded

```
public static void main(String[] args) {  
    Cat cat_1 = new Cat("Mike", 1);  
    Cat cat_2 = new Cat("Sara", 1);  
    UpperBounded<Cat> cat_up = new UpperBounded<Cat>();  
    cat_up.show(cat_1);  
    cat_up.show(cat_2);  
    Dog dog_1 = new Dog("Jen", 1);  
    Dog dog_2 = new Dog("Ken", 1);  
    UpperBounded<Dog> dog_up = new UpperBounded<Dog>();  
    dog_up.show(dog_1);  
    dog_up.show(dog_2);  
    Tiger tiger_1 = new Tiger("Jun", 4);  
    Tiger tiger_2 = new Tiger("Run", 3);  
    UpperBounded<Tiger> tiger_up = new UpperBounded<Tiger>();  
}
```

Tiger is not within bounds of type-variable T.



Generics – More than One Type

```
public class Box<T,A> {  
    private T item_t;  
    private A item_a;  
    public void set (T item_t, A item_a)  
    {  
        this.item_t = item_t;  
        this.item_a = item_a;  
    }  
    public T get_t()  
    {  
        return this.item_t;  
    }  
    public A get_a()  
    {  
        return this.item_a;  
    }  
}
```

```
public static void main(String[] args) {  
    Cat c = new Cat ("Mike", 2);  
    Dog d = new Dog ("John", 1);  
    Box<Cat,Dog> box_mixed = new Box<Cat,Dog> ();  
    box_mixed.set(c, d);  
    Cat n_c = box_mixed.get_t();  
    Dog n_d = box_mixed.get_a();  
}
```

```

public class Box<T, A> {
    private T itemT;
    private A itemA;

    // Constructor
    public Box(T itemT, A itemA) {
        this.itemT = itemT;
        this.itemA = itemA;
    }

    // Set method to set values
    public void set(T itemT, A itemA) {
        this.itemT = itemT;
        this.itemA = itemA;
    }

    // Getters for both items
    public T getT() {
        return itemT;
    }

    public A getA() {
        return itemA;
    }

    @Override
    public String toString() {
        return "ItemT: " + itemT + ", ItemA: " + itemA;
    }

    public static void main(String[] args) {
        Box<Integer, String> box1 = new Box<>(42, "Hello");
        System.out.println(box1);

        Box<Double, Character> box2 = new Box<>(3.14, 'X');
        System.out.println(box2);
    }
}

```

```

import java.util.List;

```

```

public class Box<T, A, B> {
    private T itemT;
    private A itemA;
    private B itemB;

    // Constructor
    public Box(T itemT, A itemA, B itemB) {
        this.itemT = itemT;
        this.itemA = itemA;
        this.itemB = itemB;
    }

    // Set method
    public void set(T itemT, A itemA, B itemB) {
        this.itemT = itemT;
        this.itemA = itemA;
        this.itemB = itemB;
    }

    // Getters
    public T getT() {
        return itemT;
    }

    public A getA() {
        return itemA;
    }

    public B getB() {
        return itemB;
    }

    // Complex method with List type for B
    public void printListDetails(List<B> list) {
        for (B b : list) {
            System.out.println(b);
        }
    }

    @Override
    public String toString() {
        return "ItemT: " + itemT + ", ItemA: " + itemA + ", ItemB: " + itemB;
    }

    public static void main(String[] args) {
        Box<Integer, String, List<String>> box = new Box<>(1, "Test",
        List.of("apple", "banana", "cherry"));
        System.out.println(box);
        box.printListDetails(box.getB());
    }
}

```

Generics – Upper Bounded

- **Class-Level Generics**
 - Generics allow a class to be parameterized by a type
- **Upper Bound in Generics**
 - Restricts the type parameter to a specific class or its subclasses
- **A class can define multiple type parameters**

Java Strings

- Definition:
 - Strings are objects in Java that represent a sequence of characters
- Immutability
 - Strings in Java are immutable, meaning once created, they cannot be changed.

```
String my_string = "An Example String";
```

```
String my_string = new String("An Example String");
```


Java Strings

- Since Strings are Objects, there are several built-in methods that we can use.
- `charAt()`
 - It returns the character at the specified index from a string. The index is zero-based, meaning the first character has an index of 0
 - If the specified index is negative or exceeds the string's length minus one, the method throws an `IndexOutOfBoundsException`.

```
String my_string = new String("An Example String");  
char a = my_string.charAt(7);  
System.out.println(a);  
char b = my_string.charAt(5);  
System.out.println(b);
```

*charAt (- -)
değer bulur
belirtilen sayıdaki*

Java Strings

- The `getChars(int srcBegin, int srcEnd, char[] dst, int dstBegin)` method in Java copies a substring from the source string into a destination character array
- The range is defined by `srcBegin` (inclusive) and `srcEnd` (exclusive), and the copy starts at the specified `dstBegin` index in the destination array.
- The method throws `IndexOutOfBoundsException` if the source range or destination index is invalid.

```
String my_string = new String("An Example String");  
char[] dst = new char[5];  
my_string.getChars(0, 5, dst, 0);  
System.out.println(dst);
```

An Ex

←

getChars(... , ... , char[] , ...)

Java Strings

- The `concat(String str)` method in Java concatenates the specified string `str` to the end of the current string, returning a new string that represents the combined value
- Since strings in Java are immutable, `concat` does not modify the original string but creates and returns a new string object containing the concatenated result

String
*concat(---)
→ build string*

```
String my_string = new String("An Example String");  
my_string.concat("\n").concat("Example 2").concat(" String");  
System.out.println(my_string);
```

```
String my_string = new String("An Example String");  
my_string = my_string.concat("\n").concat("Example 2").concat(" String");  
System.out.println(my_string);
```

Java Strings

- The `indexOf(String str)` method in Java is used to find the position of the first occurrence of the specified substring `str` within the string. It returns the zero-based index of the first character in the substring, or -1 if the substring is not found

```
String my_string = new String("An Example String");  
int position = my_string.indexOf("n");  
System.out.println(position);
```



```
int position_2 = my_string.indexOf("String");  
System.out.println(position_2);
```

*indexOf(...) → is indexinin yerini bulur
↳ str*

Java Strings

- The `split(String exp)` method splits a string into an array of substrings based on the specified regular expression `exp`. It separates the string wherever the `exp` matches.

split(...)

```
String my_string = new String("An Example-String");
String[] splitted = my_string.split("-");
for(int x = 0; x < splitted.length; x++)
{
    System.out.println(splitted[x]);
}
```

split ediyor ve ayırdığı parçaları arraya ekliyo.

Java Strings

- The `substring()` method is used to extract a portion of a string based on the specified indices. It does not modify the original string but returns a new string containing the selected part

```
String my_string = new String("An-Example-String");  
String first = my_string.substring(5);  
System.out.println(first);  
String second = my_string.substring(8, 15);  
System.out.println(second);
```

substring (...)

*5 dahil 5 den sonrası
ve " " sayılmıyor dahil değil*

dahil

hari

Java Strings – Example 1

```
String example = "2024-12-14,Sunny,Milan";
```

```
run:  
Year: 2024  
Month: 12  
Day: 14  
Weather: Sunny  
City: Milan  
BUILD SUCCESSFUL (total time: 0 seconds)
```

```
String[] parts = example.split(",");
```

```
String date = parts[0]; // "2024-12-14"
```

```
String weather = parts[1]; // "Sunny"
```

```
String city = parts[2]; // "Milan"
```

```
String year = date.substring(0, 4); // "2024"
```

```
String month = date.substring(5, 7); // "12"
```

```
String day = date.substring(8); // "14"
```

Java Strings – Example 1

```
int[] numbers = {2, 5, 6, 2, 7};  
String operands = "+-**";
```

run:

7

1

2

14

BUILD SUCCESSFUL (total time: 0 seconds)

```
int sum = numbers[0];  
for (int x = 0; x < operands.length(); x++)  
{  
    char op = operands.charAt(x);  
    if (op == '+')  
    {  
        sum = sum + numbers[x+1];  
        System.out.println(sum);  
    }  
    else if (op == '-')  
    {  
        sum = sum - numbers[x+1];  
        System.out.println(sum);  
    }  
    else if (op == '*')  
    {  
        sum = sum * numbers[x+1];  
        System.out.println(sum);  
    }  
    else if (op == '/')  
    {  
        sum = sum / numbers[x+1];  
        System.out.println(sum);  
    }  
}
```


QUESTION

1) Design a class named LinearEquation for a 2X2 system of linear equations:

$ax + by = e$	$x = \frac{ed - bf}{ad - bc}$	$y = \frac{af - ec}{ad - bc}$
$cx + dy = f$		

the class contains:

- private data fields **a,b,c,d,e,f**
- A constructor with the arguments for a,b,c,d,e,f
- Six getter methods for a,b,c,d,e,f
- A method named isSolvable() that returns true if $ad - bc$ is not 0 (zero)
- Methods getX() and getY() that return the solution for the equation

An example run:

$$\begin{aligned}2x + 3y &= 8 \\4x - y &= 2\end{aligned}$$

```
run:
Equation is solvable: true
X is 1.0
Y is 2.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

ANKARA YILDIRIM BEYAZIT UNIVERSITY

SENG 201 OBJECT ORIENTED PROGRAMMING

LAB 3

09.10.2024

QUESTIONS

- 1) Write a program to show the distinct values in the array.

```
Original Array
12 3 4 5 6 1 3 1 4
Distinct Values
1 3 4 5 6 12 BUILD SUCCESSFUL (total time: 0 seconds)
```

- 2) Given a 2D array, locate the position of the smallest element.

```
3.0 5.6 7.2
4.6 1.3 8.7
5.0 9.1 3.2
7.9 5.1 2.7
The location of the smallest element is (1, 1)
```

ANKARA YILDIRIM BEYAZIT UNIVERSITY

SENG 201 OBJECT ORIENTED PROGRAMMING

LAB 4

16.10.2024

QUESTIONS

- 1) An Emirp (prime spelled backward) is a nonpalindromic prime number whose reversal is also a prime. For example, 17 is a prime and 71 is a prime. So, 17 and 71 are emirps. Write a program that displays the first 100 emirps.

```
run:
```

```
13
```

```
17
```

```
31
```

```
37
```

```
71
```

```
73
```

```
79
```

```
97
```

```
107
```

```
113
```

```
149
```

```
157
```

```
167
```

```
.
```

```
.
```

```
.
```

```
.
```

```
.
```

QUESTION

1) Design a class named LinearEquation for a 2X2 system of linear equations:

$ax + by = e$	$x = \frac{ed - bf}{ad - bc}$	$y = \frac{af - ec}{ad - bc}$
$cx + dy = f$		

the class contains:

- private data fields **a,b,c,d,e,f**
- A constructor with the arguments for a,b,c,d,e,f
- Six getter methods for a,b,c,d,e,f
- A method named isSolvable() that returns true if $ad - bc$ is not 0 (zero)
- Methods getX() and getY() that return the solution for the equation

An example run:

$$\begin{aligned} 2x + 3y &= 8 \\ 4x - y &= 2 \end{aligned}$$

```
run:
Equation is solvable: true
X is 1.0
Y is 2.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

ANKARA YILDIRIM BEYAZIT UNIVERSITY

SENG 201 OBJECT ORIENTED PROGRAMMING

LAB 6

20.11.2024

QUESTION

- 1) Design a class structure for three different types of vehicles; Truck, Car, and Motorcycle. Common vehicle attributes are make, model, model year, fuel type, engine size, weight, maximum speed and efficiency multiplier. A truck has a special variable for cargo capacity, a car has a special variable for seat count and a motorcycle has no special variables.
- Efficiency multiplier will be determined at initialization; Diesel fuel has 0.9 efficiency multiplier and Gasoline fuel has 1.1 efficiency multiplier.
 - All 3 vehicle types have a common way to calculate fuel usage which is $\left(1.45 + \frac{\text{engine size}}{1000} * \frac{\text{weight}}{1000}\right) * \text{efficiency multiplier}$
 - For truck, this formula will be multiplied by cargo capacity / 35000 and for car, this formula will be multiplied by 0.9 + (seat count / 5). Motorcycle will use the formula without any extra multipliers directly.
 - You will also calculate travelled distance based on the maximum speed of the vehicle using a given time in hours.
 - Write getter methods for all data members of your classes.
 - Display the car information, fuel usage, and distance as shown in the example run.
 - You do not need to take any inputs from the user.

An example run: (time is given as 0.65 hours, cargo capacity is 35000, car has 5 seats)

Type	Engine Size	Weight
Truck	7990	4830
Car	1396	1215
Motorcycle	649	208

```
run:
Truck Model: Isuzu FTR 150-260 Auto 2022 Diesel
Fuel Usage: 15.444655714285716 l/100 km
Distance Traveled: 87.75 kilometers
Max Speed: 135.0 kph

Car Model: Volkswagen Golf 2019 Gasoline
Fuel Usage: 6.5754326 l/100 km
Distance Traveled: 132.6 kilometers
Max Speed: 204.0 kph

Motorcycle Model: Honda CB 650 F 2014 Gasoline
Fuel Usage: 1.7434912 l/100 km
Distance Traveled: 163.15 kilometers
Max Speed: 251.0 kph
BUILD SUCCESSFUL (total time: 0 seconds)
```



ANKARA YILDIRIM BEYAZIT UNIVERSITY
SENG 201 OBJECT ORIENTED PROGRAMMING
QUESTION

25/11/2024

- 1) Create a ComplexNumber class to represent complex numbers and perform basic operations such as addition, subtraction, and multiplication. Implement the following functionalities
 - a. Define the ComplexNumber class:
 - i. Two private attributes: **real** and **imaginary** (both of type double).
 - ii. A parameterized constructor to initialize the real and imaginary parts
 - iii. A default constructor that initializes both real and imaginary parts to 0
 - b. Implement the following methods:
 - i. **add (ComplexNumber other)**: Adds the current complex number with another complex number and returns the resulting complex number.
 - ii. **subtract (ComplexNumber other)**: Subtracts another complex number from the current complex number and returns the resulting complex number.
 - iii. **multiply (ComplexNumber other)**: Multiplies the current complex number with another complex number and returns the resulting complex number.
 - iv. **toString()**: Returns a string representation of the complex number in the format "a + bi" (e.g., 5.0 + 3.0i)
 - c. Write a Main class to test the implementation
 - i. Create two complex numbers (e.g., 5 + 3i and 2 + 4i).
 - ii. Perform and display the results of addition, subtraction, and multiplication of the two complex numbers

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    Complex c1 = new Complex(1,4);  
    Complex c2 = new Complex(2,6);  
  
    Complex result_sum = c1.add(c2);  
    Complex result_sub = c1.sub(c2);  
    Complex result_mul = c1.mul(c2);  
    System.out.println(result_sum);  
    System.out.println(result_sub);  
    System.out.println(result_mul);  
}
```

```
run:  
(3.0+10.0i)  
(-1.0-2.0i)  
(-22.0+14.0i)
```

